Pyramids

Pablo Mesejo
pmesejo@go.ugr.es
Universidad de Granada
Departamento de Ciencias de la Computación e Inteligencia Artificial



Index

- Image downsampling and aliasing
- Image pyramids:
  - Gaussian pyramids
  - Laplacian pyramids
  - Image pyramids applications
- Thinking in frequency: Fourier domain vs Space domain
- Upsampling and interpolation

En este bloque de diapositivas, y ya como parte final del tema 2, dedicado a la representación y el filtrado de imágenes, vamos a abordar cómo se reduce el tamaño de una imagen y qué es el aliasing.

Estudiaremos las pirámides de imágenes, principalmente las pirámides gaussianas y laplacianas, así como algunas de sus aplicaciones.

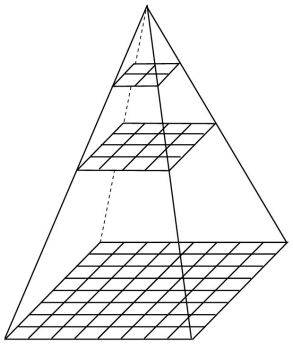
También aprenderemos a pensar un poco en términos de frecuencia. Contrastaremos el dominio de Fourier con el dominio espacial, lo que nos ayudará a comprender mejor qué queremos decir cuando afirmamos que un filtro atenúa las altas o las bajas frecuencias.

Finalmente, veremos cómo se puede aumentar el tamaño de una imagen y qué papel desempeña la interpolación.

Dealing with Scale: Pyramids

Scale-space

- We cannot process images at a single scale.
  - "No single scale is categorically correct: the physical processes that generate signals such as images act at a variety of scales, and none is intrinsically more interesting or important than another." (Witkin, 1984)
- The most natural structure to operate on multiple scales: a pyramid
  - **Mixing local features from multiple scales** is an effective approach and is consistent with the Human Visual System (HVS)
- We'll come back to these ideas when discussing interest point detection and feature description (e.g. SIFT).



Witkin, A. (1984). *Scale-space filtering: A new approach to multi-scale description*. In *ICASSP'84. IEEE International Conference on Acoustics, Speech, and Signal Processing* (Vol. 9, pp. 150-153).
Lindeberg, T. (1988). *On the construction of a scale-space for discrete images*. KTH Royal Institute of Technology.
Lindeberg, T. (1994). *Scale-space theory: A basic tool for analyzing structures at different scales*. *Journal of applied statistics* 21.1-2 (pp. 225-270).

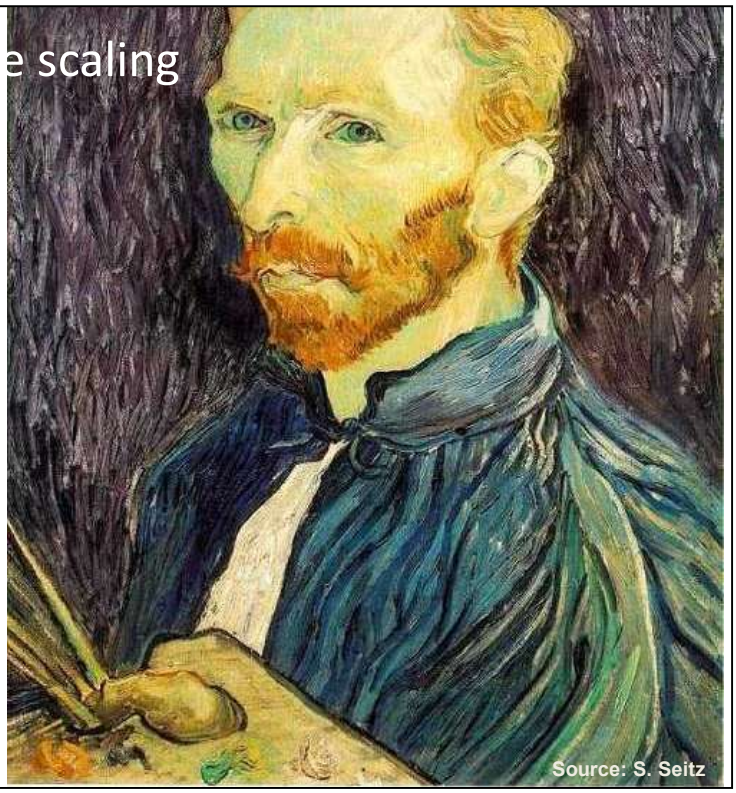
Lo primero que debemos asumir es que no podemos procesar imágenes considerando una única escala, porque no existe una única escala que sea categóricamente correcta para todos los elementos de una imagen.

Partiendo de esta idea, una estructura natural para representar y procesar múltiples escalas es una pirámide de imágenes. Esto nos permite combinar características locales a distintas escalas y, en cierta medida, aproximarnos al modo en que opera el sistema visual humano.

Volveremos a estas ideas más adelante cuando estudiemos la detección de puntos de interés y la descripción de características, por ejemplo, con el algoritmo SIFT.

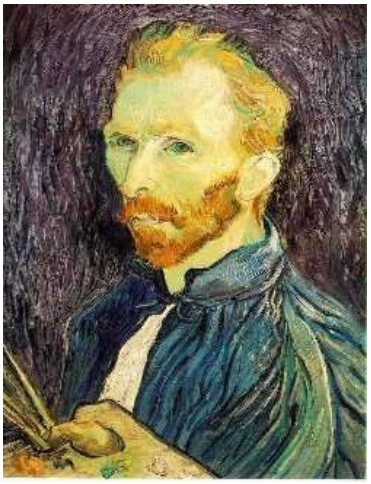
Image scaling

This image is too big to fit on the screen. How can we generate a half-sized version?



Imaginemos que queremos redimensionar esta imagen de Vincent Van Gogh. Ahora mismo es demasiado grande para caber en la pantalla, así que queremos obtener una versión que tenga la mitad de ancho y la mitad de alto.

Image sub-sampling





1/4



1/8

Throw away every other row and column to create an image with ½ rows and ½ columns
- called ***image sub-sampling***

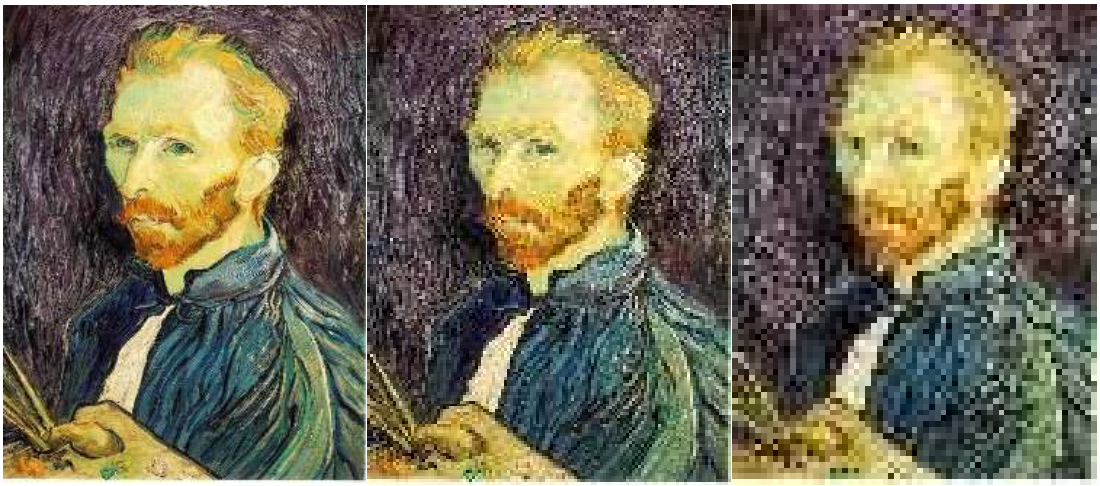
Source: S. Seitz

La forma más inmediata de hacerlo sería, sencillamente, eliminar una de cada dos filas y una de cada dos columnas, por ejemplo, las filas y columnas pares o las impares. De esa manera nos quedaríamos con la mitad de las filas y la mitad de las columnas.

Si quisiéramos volver a reducir a la mitad las dimensiones de la imagen, podríamos repetir exactamente el mismo procedimiento, y así sucesivamente.

Aquí vemos tres ejemplos: a la izquierda, una imagen cuyo ancho y alto son la mitad de los de la original; en el centro, una imagen cuyo ancho y alto son una cuarta parte; y a la derecha, una imagen cuyo ancho y alto son una octava parte de los de la imagen original.

Image sub-sampling



1/2 1/4 (2x zoom) 1/8 (4x zoom)

Why does this look so crufty?

At the frequency level, what are we doing here? 🤔

Source: S. Seitz

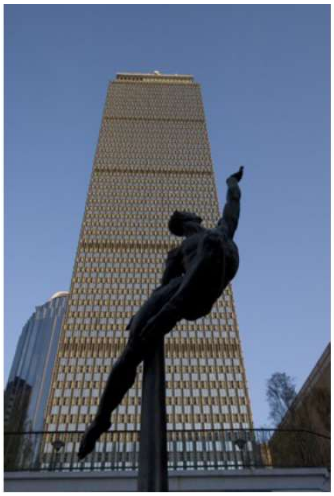
Si ahora hacemos un poco de zoom sobre cada una de estas imágenes submuestreadas para observarlas con más detalle, veremos que, a medida que reducimos más y más su tamaño, la apariencia se va degradando y empiezan a aparecer artefactos: saltos, patrones extraños o estructuras que no estaban presentes en la imagen original.

Aquí aparece una idea clave. Si eliminamos píxeles sin realizar ningún procesamiento previo, podemos estar introduciendo artefactos en la imagen.

Es interesante porque, intuitivamente, uno podría pensar: «Si estoy quitando píxeles, estoy eliminando información; no estoy introduciendo nada nuevo». Sin embargo, el resultado puede contener estructuras aparentes que no existían en la imagen original. En cierto sentido, es como si estuviésemos “inyectando altas frecuencias” (es decir, es como si estuviésemos introduciendo ruido).

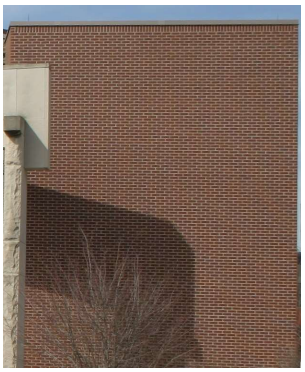
Y aquí viene la pregunta: ¿cómo evitamos ese problema? ¿Cómo podemos atenuar esas altas frecuencias? Antes de submuestrear debemos alisar la imagen, es decir, aplicar un filtro paso bajo que elimine las frecuencias que la nueva resolución ya no va a poder representar correctamente.

Image sub-sampling





Source: F. Durand





Source: https://en.wikipedia.org/wiki/Aliasing

Aliasing artifacts usually occur in the form of Moiré patterns (https://mathworld.wolfram.com/MoirePattern.html)

8 of 90

Los artefactos que aparecen nos resultan bastante familiares. Este fenómeno se denomina aliasing. El término alias hace referencia precisamente a que, después del muestreo, frecuencias diferentes pueden volverse indistinguibles y aparecer representadas como otras frecuencias distintas de las originales.

En esta diapositiva podéis ver la imagen original a la izquierda y una versión con aliasing a la derecha. Podemos observar distintos artefactos y, en determinadas situaciones, estos aparecen en forma de los conocidos patrones de Moiré.

Los patrones de Moiré son una manifestación especialmente visible del aliasing espacial y aparecen sobre todo cuando tenemos estructuras periódicas o repetitivas. Es algo parecido a lo que ocurre cuando superponemos dos mosquiteras: pueden aparecer patrones de rayas o bandas de una escala mucho mayor que, en realidad, no estaban presentes en ninguna de las dos estructuras por separado.

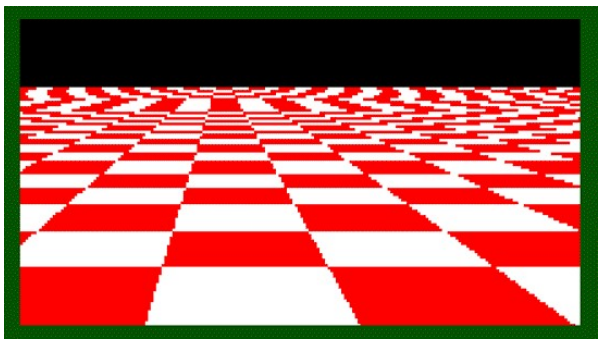
En el caso de las imágenes, esto sucede cuando intentamos muestrear una estructura que contiene frecuencias espaciales demasiado altas para la resolución disponible. Esas frecuencias ya no pueden representarse correctamente y terminan apareciendo como otras frecuencias espurias, que pueden dar lugar a esos patrones visuales.

Esto puede ocurrir principalmente de dos maneras. La primera es durante la propia adquisición de la imagen. Por ejemplo, una fachada puede contener una estructura repetitiva tan fina que la resolución espacial del sensor no sea suficiente para

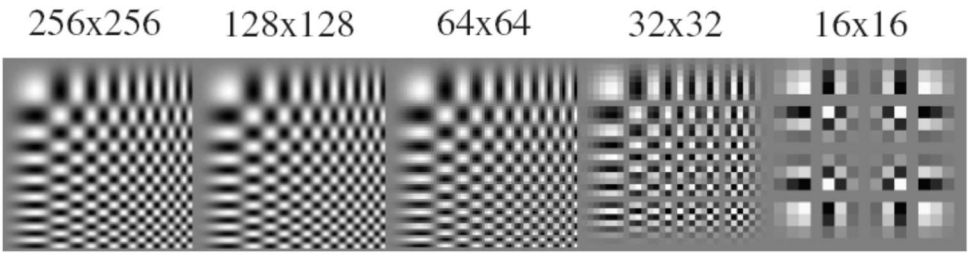
representarla correctamente. En ese caso, el aliasing aparece ya al pasar de la escena continua a la imagen digital.

La segunda posibilidad aparece cuando ya tenemos una imagen digital y la submuestreamos de manera demasiado agresiva sin aplicar previamente un filtro paso bajo. Al reducir la frecuencia de muestreo, algunas frecuencias que antes podían representarse correctamente dejan de poder hacerlo, produciendo aliasing y, en determinadas situaciones, patrones de Moiré.

Even worse for synthetic images



Disintegrating textures



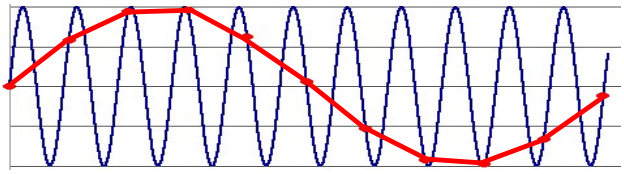
9 of 90

Este defecto, que sin duda puede resultar molesto en imágenes reales, puede ser todavía más llamativo en imágenes sintéticas. Como podéis ver en esta diapositiva, al submuestrear este tipo de imágenes las texturas pueden llegar incluso a desestructurarse por completo.

En la imagen superior tenemos una especie de tablero de ajedrez con casillas blancas y rojas sometido a una transformación proyectiva. A medida que avanzamos hacia el fondo, las casillas deberían hacerse cada vez más pequeñas. Sin embargo, llega un punto en el que el patrón deja de representarse correctamente.

En la parte inferior ocurre algo parecido. Partimos de una imagen sintética muy sencilla de 256×256 píxeles y vamos reduciendo progresivamente su resolución. Cuando llegamos a 16×16 píxeles el patrón resultante ha cambiado de forma radical respecto al original.

Aliasing



- **Occurs when <u>your sampling rate is not high enough</u> to capture the amount of detail in your signal/image**
- **Can give you the wrong signal**—an *alias*

- To do sampling right, need to understand the structure of your signal
  - We need to think in frequency terms!

10 of 90

En esta diapositiva vemos el efecto del aliasing sobre una señal unidimensional, lo que nos permite entenderlo de una manera muy intuitiva.

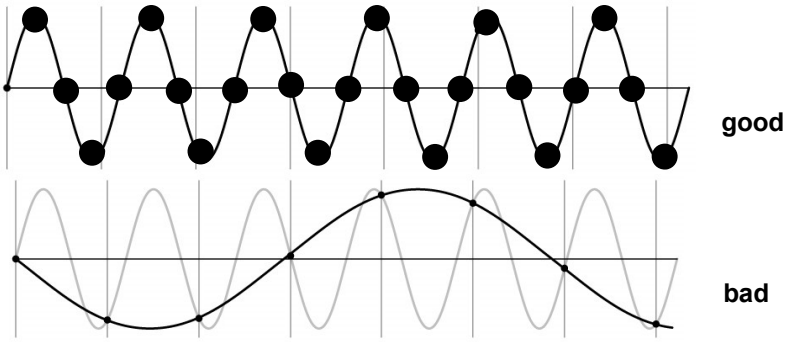
Tenemos una señal sinusoidal original, pero nuestra frecuencia de muestreo no es lo suficientemente alta como para capturar correctamente todas sus variaciones. Como consecuencia, las muestras obtenidas también son compatibles con otra señal diferente, de menor frecuencia.

Aquí vemos precisamente eso: la señal azul es la original, pero, debido a que la tasa de muestreo es insuficiente, no podemos reconstruirla de manera unívoca a partir de las muestras. En su lugar, esas mismas muestras pueden interpretarse como pertenecientes a la señal roja, que actúa como un alias de la señal original.

Aliasing

- **Occurs when <u>your sampling rate is not high enough</u> to capture the amount of detail in your signal/image**
- **Can give you the wrong signal**—an *alias*

- To avoid aliasing:
  - Sample more often!!!
    - sampling rate > 2 * max frequency in the image
      - said another way:
        > two samples per cycle
    - This minimum sampling rate is called the **Nyquist rate**



11 of 90

Para evitar el aliasing, tenemos fundamentalmente dos opciones. La primera, que es la que vemos en esta diapositiva, consiste en muestrear con mayor frecuencia. Podríamos interpretarlo, en cierto modo, como mejorar la tecnología o el hardware de adquisición.

Lo que necesitamos es que la frecuencia de muestreo sea mayor que dos veces la frecuencia máxima presente en la señal. Dicho de otro modo, necesitamos más de dos muestras por ciclo. Ese valor, dos veces la frecuencia máxima, se denomina tasa de Nyquist.

En esta misma diapositiva vemos, abajo a la derecha, una señal que ha sido muestreada con una frecuencia insuficiente, por debajo de esa tasa mínima, y por tanto aparece aliasing. En cambio, en la parte superior, donde la frecuencia de muestreo supera la tasa de Nyquist, sí podemos reconstruir correctamente la señal.

Aliasing

- **Occurs when your sampling rate is not high enough to capture the amount of detail in your signal/image**
- **Can give you the wrong signal**—an *alias*

- To avoid aliasing:
  - Sample more often!!!
    - sampling rate > 2 * max frequency in the image
      - said another way: > two samples per cycle
    - This minimum sampling rate is called the **Nyquist rate**
  - Reduce high-frequencies in the image!!!
    - What remedy do we know to reduce the presence of noise?
      Apply a smoothing (low-pass) filter

12 of 90

La segunda estrategia, que es en la que nos vamos a centrar en esta asignatura, consiste sencillamente en reducir las altas frecuencias presentes en la imagen.

Por tanto, tenemos dos estrategias principales. La primera sería muestrear con mayor frecuencia, que podríamos considerar la opción hardware. La segunda consiste en reducir las componentes de alta frecuencia antes de realizar el muestreo o submuestreo.

¿Y cuál es el mecanismo que ya conocemos para reducir las altas frecuencias? Aplicar un filtro de suavizado, es decir, un filtro paso bajo.

Aliasing – helicopter blades



https://www.youtube.com/watch?v=yr3ngmRuGUc

13 of 90

En esta diapositiva podéis ver un ejemplo extremo de aliasing temporal. La frecuencia de adquisición de frames del vídeo no es suficientemente alta para capturar correctamente el movimiento de las aspas del helicóptero.

De hecho, las aspas parecen prácticamente inmóviles, de manera que da la impresión de que el helicóptero está levitando en el aire. Esto ocurre porque la frecuencia de muestreo temporal está relacionada con la frecuencia de rotación de las aspas, de modo que cada frame las captura en una posición muy similar.

El movimiento real es mucho más rápido, pero debido al aliasing temporal observamos un movimiento aparente completamente distinto.

Aliasing in practice





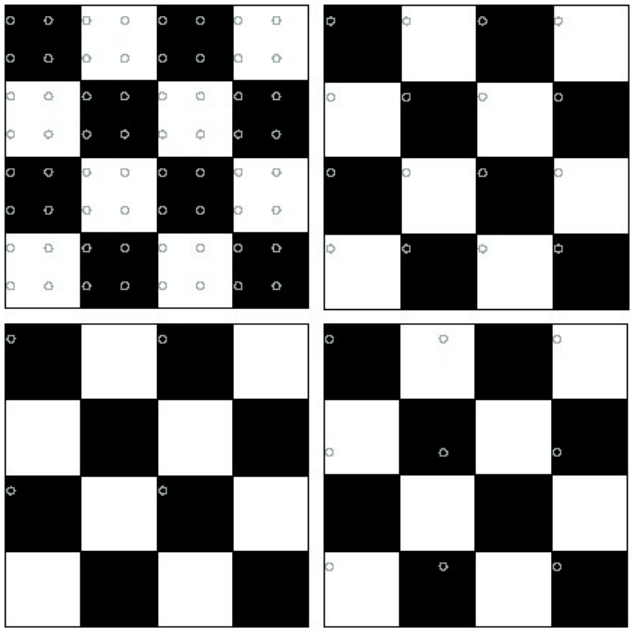
- Sub-sampling may be dangerous….
- Characteristic errors (distortion artifacts) may appear:
  - “Wagon wheels rolling the wrong way in movies”
  - “Checkerboards disintegrate in ray tracing”
  - “Striped shirts look funny on color television”

En términos prácticos, el aliasing aparece en numerosas situaciones y da lugar a artefactos bastante característicos.

Por ejemplo, podemos encontrar patrones de ajedrez o texturas periódicas que se desestructuran, prendas de ropa con rayas que adquieren una apariencia extraña en televisión, o incluso ruedas que parecen girar en el sentido contrario al real, el conocido wagon-wheel effect.

Todos estos son ejemplos distintos de un mismo fenómeno: la frecuencia de muestreo no es suficiente para representar correctamente las variaciones espaciales o temporales presentes en la señal.

Nyquist limit – 2D example



Good sampling

Bad sampling

15 of 90

En esta diapositiva tenéis un ejemplo visual muy sencillo, en 2D, de la diferencia entre un buen muestreo y un mal muestreo.

La fila superior muestra un ejemplo de muestreo adecuado, que sí nos permite recuperar correctamente el patrón blanco y negro del tablero de ajedrez.

En la fila inferior, en cambio, la frecuencia de muestreo es insuficiente. De hecho, en el ejemplo de abajo a la izquierda, las muestras obtenidas serían compatibles con una imagen completamente negra. Es decir, habríamos perdido por completo la estructura original del tablero.

Aliasing

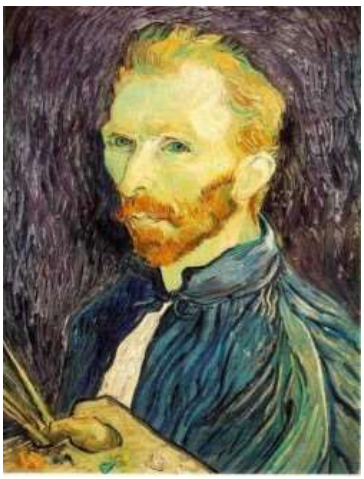
- When downsampling by a factor of two
  - Original image has frequencies that are too high
- How can we fix this?
  - We need spatial anti-aliasing!

16 of 90

Por lo tanto, a modo de resumen, cuando submuestreamos una imagen por un factor de dos, es decir, cuando reducimos a la mitad el número de filas y el número de columnas, algunas de las frecuencias presentes en la imagen original pueden ser demasiado altas.

¿Cómo podemos evitar el aliasing? Aplicando previamente antialiasing espacial: suavizamos la imagen mediante un filtro paso bajo para eliminar/atenuar las frecuencias que la nueva resolución ya no puede representar correctamente.

Gaussian pre-filtering



Gaussian 1/2



G 1/4



G 1/8

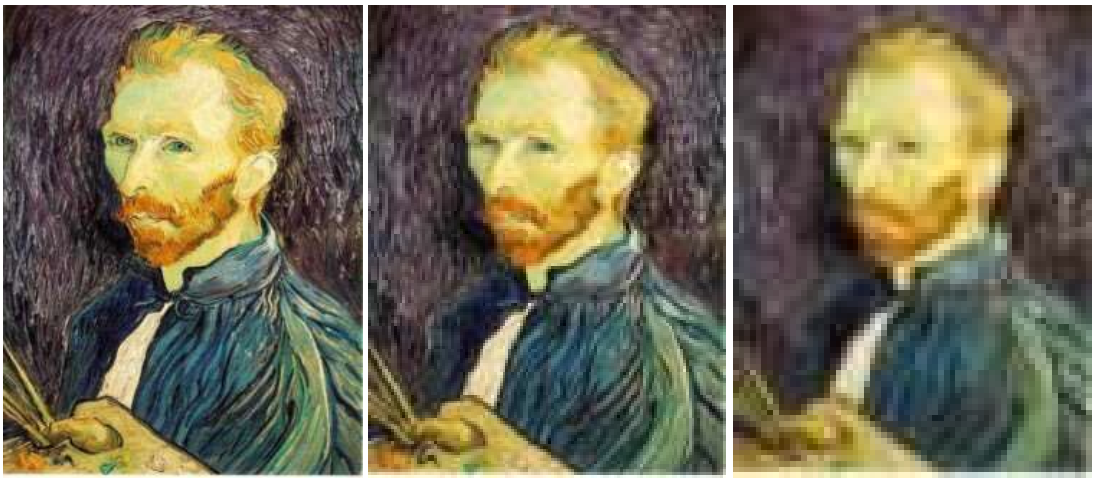
- **Solution:** filter the image, *then* subsample

Source: S. Seitz

17 of 90

De modo que, en definitiva, lo que tenemos que hacer antes de submuestrear es aplicar un filtro paso bajo. En nuestro caso utilizaremos un suavizado gaussiano, que es además el procedimiento que emplearemos para construir la pirámide gaussiana. El filtro gaussiano suele ser preferible porque tiene una respuesta en frecuencia más suave y se comporta mejor que un filtro de medias (como ya hemos podido observar en diapositivas anteriores). Primero suavizamos la imagen y después reducimos su resolución. A continuación, volvemos a suavizar la imagen resultante y volvemos a submuestrear, y así sucesivamente.

Subsampling with Gaussian pre-filtering



Gaussian 1/2 G 1/4 G 1/8

- **Solution:** filter the image, *then* subsample

Source: S. Seitz

En esta diapositiva y en la siguiente podemos comparar el resultado de aplicar primero un filtro gaussiano y después submuestrear, frente a submuestrear directamente sin realizar ningún prefiltrado.

Independientemente de otras consideraciones, podemos observar que cuando el submuestreo se realiza después de aplicar un prefiltrado gaussiano, el resultado visual es claramente mejor y aparecen menos artefactos de aliasing.

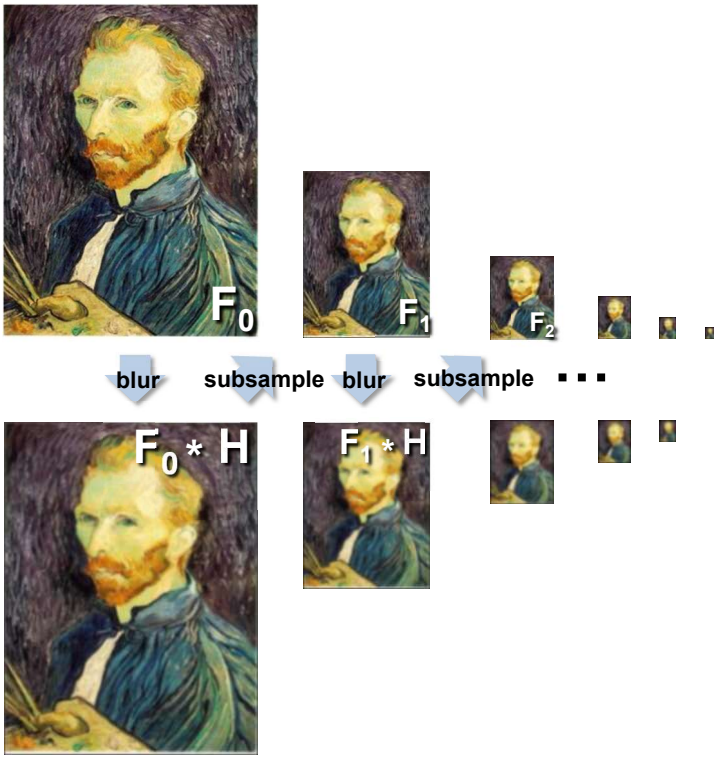
Compare with...



1/2 **1/4 (2x zoom)** **1/8 (4x zoom)**

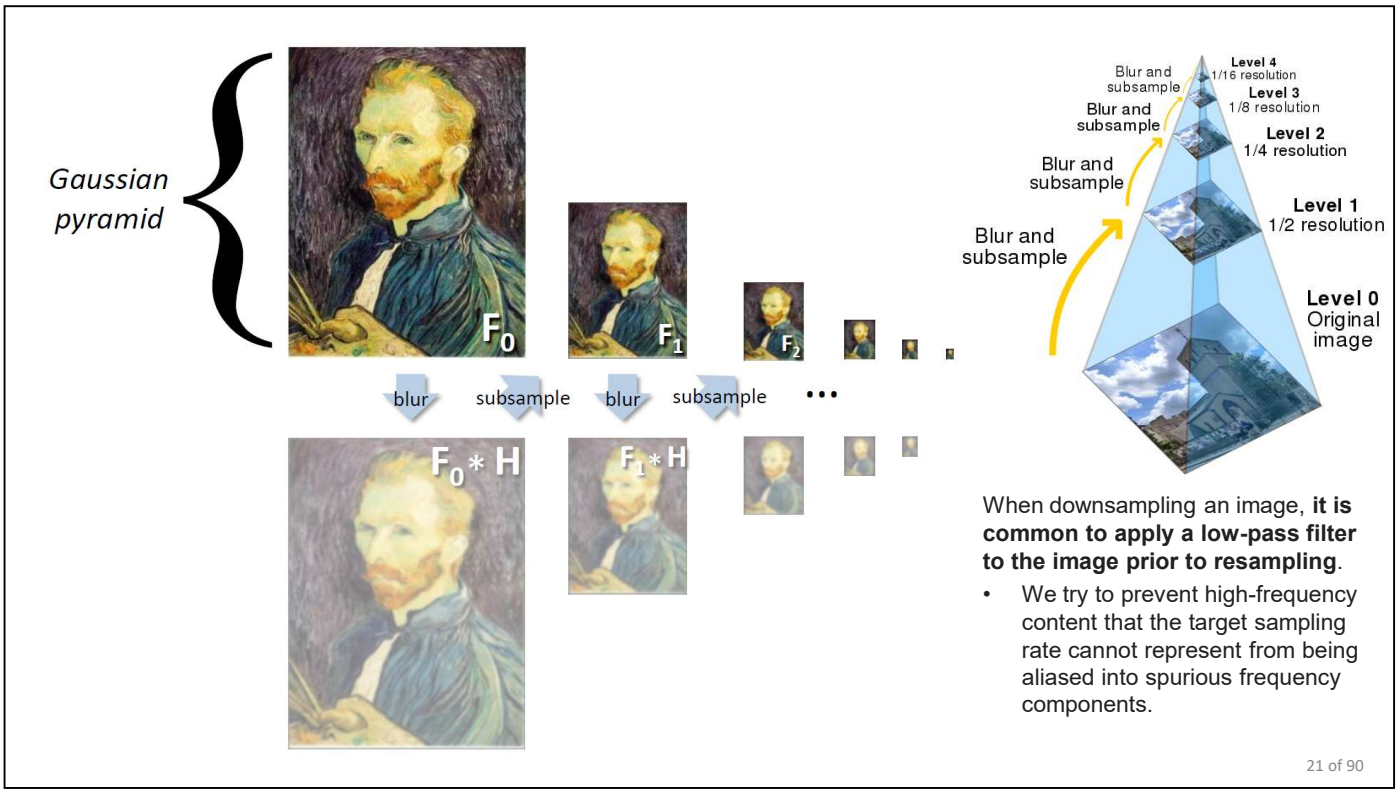
Source: S. Seitz

- **Solution:** filter the image, *then* subsample



20 of 90

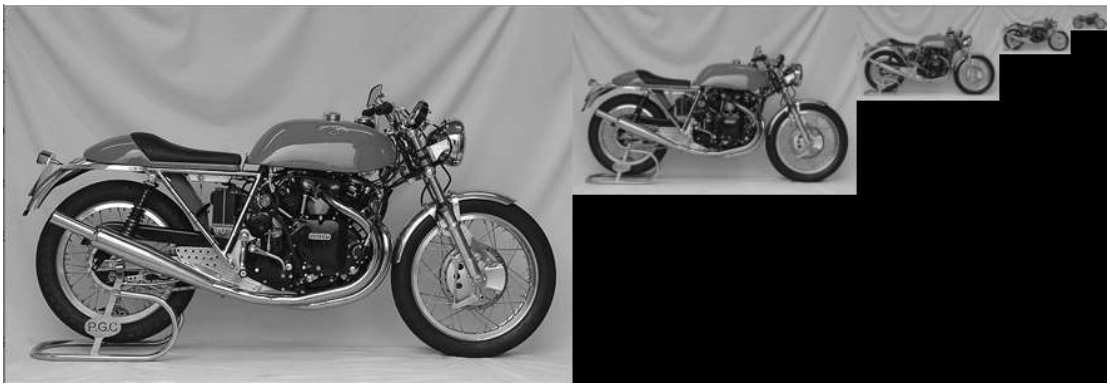
Y en esta diapositiva podemos ver el procedimiento completo. Partimos de la imagen F0, que es la imagen original. Primero la convolucionamos con el filtro gaussiano H y, a continuación, submuestreamos para obtener F1. Después repetimos exactamente el mismo proceso: suavizamos F1 mediante la convolución con el filtro gaussiano H, submuestreamos y obtenemos F2, y así sucesivamente.



La parte superior de este proceso, donde tenemos F0, F1, F2, F3, etcétera, constituye la pirámide gaussiana.

Es importante remarcar una convención que vamos a utilizar en esta asignatura. Cuando hablamos, por ejemplo, de una pirámide gaussiana con cuatro niveles, estamos considerando además el nivel 0, que corresponde a la imagen original. De modo que, si en las prácticas os pido una pirámide gaussiana con cinco niveles, tendréis que visualizar seis imágenes: la imagen original más los cinco niveles obtenidos sucesivamente.

Gaussian pyramids [Burt and Adelson, 1983]



- How much space does a Gaussian pyramid take compared to the original image? 🤔

$$1 + \frac{1}{4} + \frac{1}{16} + \cdots + \left(\frac{1}{4}\right)^n$$

As n approaches infinity, the total space approaches 4/3 times the original image size.

22 of 90

En esta diapositiva, por ejemplo, tenéis una pirámide gaussiana con cinco niveles. A la izquierda, la imagen más grande corresponde a la imagen original sin procesar y, a medida que nos desplazamos hacia la derecha, vamos aplicando sucesivamente suavizado y submuestreo.

Una pregunta pertinente es cuánto espacio ocupa una pirámide gaussiana en comparación con la imagen original. Si consideramos que la imagen original ocupa 1, el siguiente nivel ocupa un cuarto, porque hemos reducido a la mitad el número de filas y a la mitad el número de columnas. El siguiente nivel ocupa un dieciseisavo de la imagen original, y así sucesivamente.

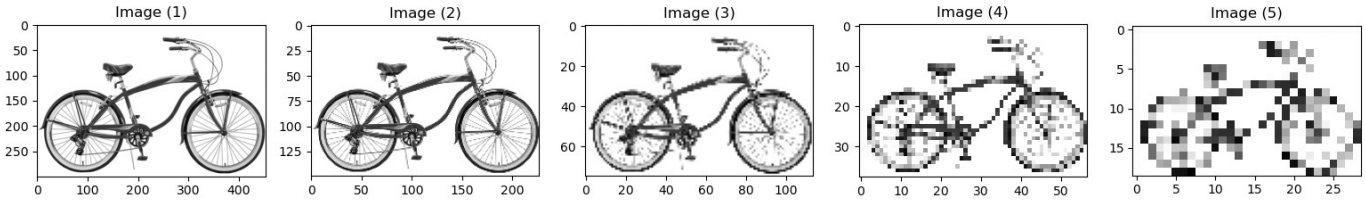
Por tanto, tenemos 1 más 1/4 más 1/16 más 1/64, etcétera. Si prolongamos idealmente esta serie hasta infinitos niveles, su suma converge a 4/3.

Es decir, una pirámide gaussiana completa ocuparía, como máximo, aproximadamente un 33 % más que la imagen original. Como vemos, el coste adicional de almacenamiento es relativamente pequeño.

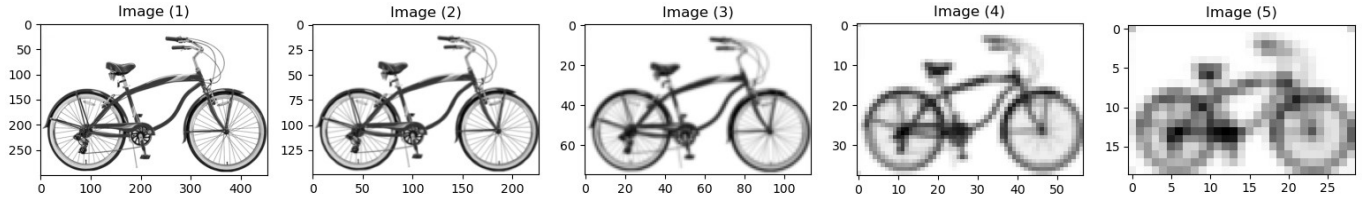
Gaussian Pyramids

- Effect of smoothing when creating the pyramid

No Gaussian smoothing. Just subsampling, keeping the even rows/columns (aliasing):



Including Gaussian smoothing:



23 of 90

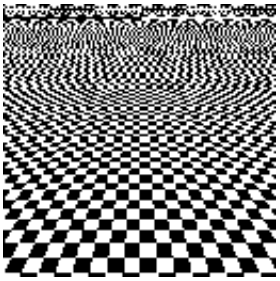
Y en esta diapositiva podemos observar el efecto práctico de aplicar el suavizado durante la construcción de la pirámide.

Imaginemos que no aplicamos ningún prefiltrado gaussiano y que sencillamente submuestreamos conservando una fila sí y una fila no, una columna sí y una columna no. En ese caso podemos observar que ya en la tercera imagen de la primera fila comienzan a aparecer artefactos propios del aliasing. En la cuarta y en la quinta imagen estos artefactos se hacen todavía más evidentes.

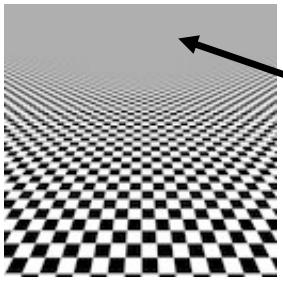
En cambio, cuando incluimos el prefiltrado gaussiano antes de cada submuestreo, observamos que la tercera imagen presenta muchos menos artefactos y mantiene una apariencia más natural. Evidentemente, a medida que seguimos suavizando y submuestreando, la bicicleta se va difuminando progresivamente y perdemos detalle, pero evitamos esos artefactos bruscos característicos del aliasing.

Back to the checkerboard

- What should happen when you make the checkerboard smaller and smaller?



Naïve subsampling



Proper prefiltering ("antialiasing")

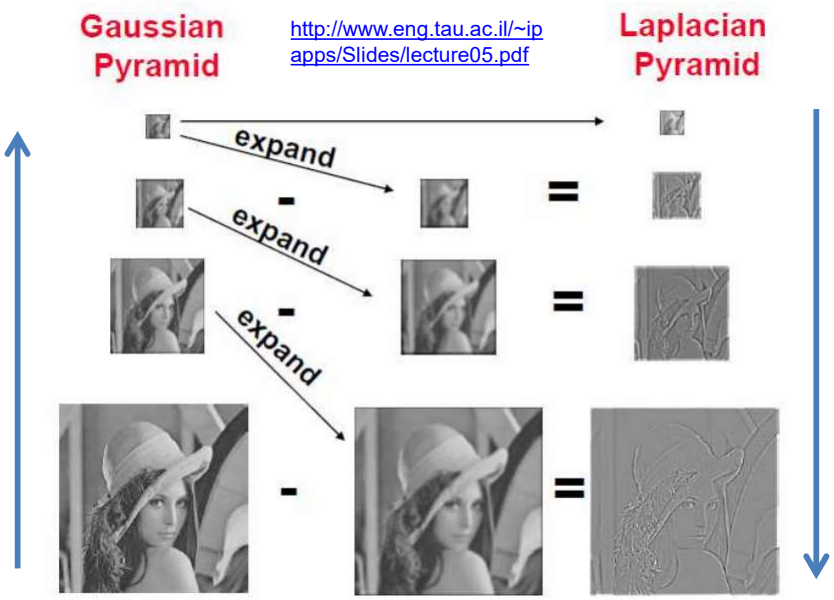
Image turns grey! (Average of black and white squares, because each pixel contains both)

24 of 90

Aquí mostramos otro ejemplo, esta vez con un patrón de tablero de ajedrez. Cuando aplicamos un submuestreo ingenuo, es decir, sin ningún tipo de prefiltrado, observamos de nuevo que el patrón se desestructura por completo. Llega un momento en el que la textura original se transforma aparentemente en otra distinta debido al aliasing.

¿Qué debería ocurrir si hacemos el tablero de ajedrez cada vez más pequeño aplicando correctamente un filtro de antialiasing antes de cada submuestreo? A medida que disminuye la resolución, cada píxel representa una región cada vez mayor de la imagen original y combina información procedente de zonas blancas y negras. Por tanto, llega un momento en el que esas variaciones dejan de poder resolverse y el patrón converge hacia un tono gris, aproximadamente el valor medio de los cuadrados blancos y negros.

Laplacian Pyramids



1) We start from the smallest image generated by the Gaussian Pyramid.
2) We expand it, we compute the difference with the next level in the Gaussian Pyramid, and we get the L_i Level of the Laplacian Pyramid.
3) We move to the next Gaussian level, we expand it, compute the difference, and we already have level L_{i-1} in the Laplacian Pyramid.
4) And so on...

25 of 90

Y ahora, después de toda esta explicación, podemos llegar a la pirámide laplaciana, que es lo que visualizamos en esta diapositiva.

A la izquierda tenemos la construcción de la pirámide gaussiana, que va de abajo hacia arriba: partimos de la imagen original y vamos obteniendo imágenes progresivamente más suavizadas y de menor resolución. El procedimiento consiste en suavizar, submuestrear, suavizar, submuestrear, y así sucesivamente. En este caso tenemos una pirámide gaussiana de tres niveles. Recordemos que, siguiendo la convención que vamos a utilizar en esta asignatura, una pirámide gaussiana de tres niveles contiene cuatro imágenes, porque incluimos también la imagen original como nivel 0.

Para construir la pirámide laplaciana partimos de la cima de la pirámide gaussiana. La imagen más pequeña se conserva directamente, de modo que la cima de la pirámide gaussiana y la cima de la pirámide laplaciana son exactamente la misma imagen.

A partir de ahí descendemos por la pirámide. Tomamos el nivel inmediatamente superior de la pirámide gaussiana, lo expandimos hasta recuperar el tamaño del nivel anterior, y restamos esa imagen expandida al nivel correspondiente de la pirámide gaussiana. De esta forma obtenemos la información de detalle que se había perdido al pasar de un nivel al siguiente.

Repetimos exactamente el mismo procedimiento con los siguientes niveles: expandimos el nivel superior y lo restamos del nivel inmediatamente inferior de la

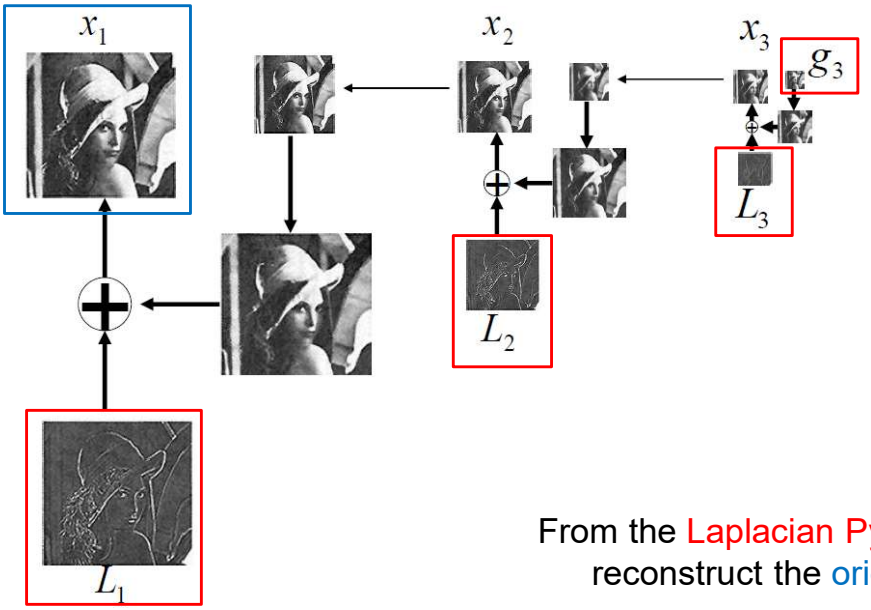
pirámide gaussiana. Así obtenemos sucesivamente las distintas bandas de detalle hasta llegar a la base.

En definitiva, la pirámide gaussiana contiene la imagen original y versiones progresivamente suavizadas y de menor resolución. La pirámide laplaciana, en cambio, contiene principalmente información de detalle asociada a diferentes escalas, excepto en su cima, donde conservamos directamente la imagen más pequeña de la pirámide gaussiana.

La pirámide laplaciana recibe ese nombre porque sabemos que la Laplaciana de la gaussiana puede aproximarse mediante una diferencia de gaussianas, y esa idea está estrechamente relacionada con lo que hacemos en esta construcción: obtener, a distintas escalas, la diferencia entre una imagen y una versión más suavizada de la misma.

Laplacian Pyramids

- We are able to reconstruct x_1 from L_1, L_2, L_3 and g_3



From the Laplacian Pyramid, we can reconstruct the original image

26 of 90

Algo muy interesante de la pirámide laplaciana es que nos permite reconstruir exactamente la imagen original, siempre que utilicemos en la reconstrucción la misma operación de expansión que se utilizó al construirla.

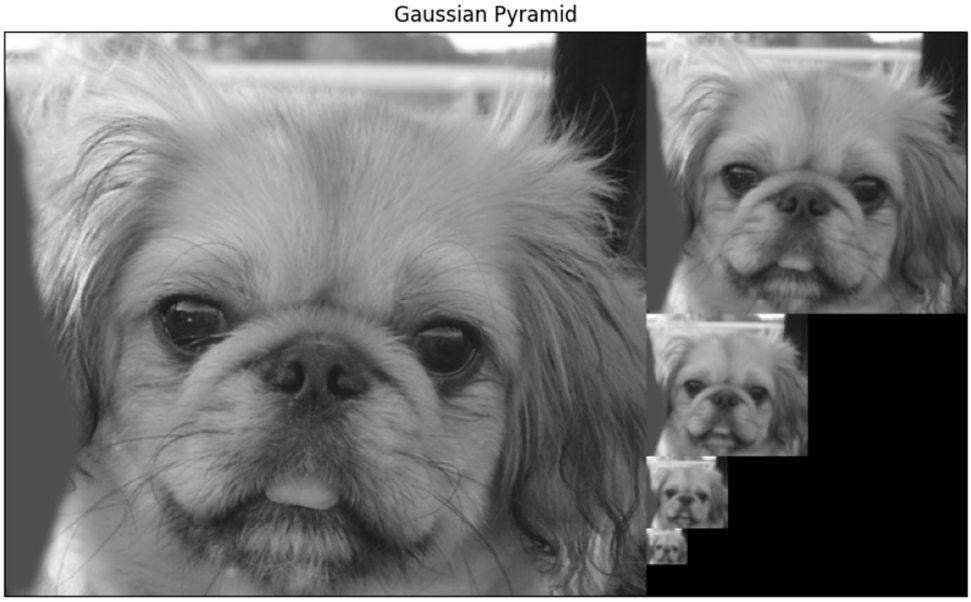
Y, si nos paramos a pensarlo, tiene mucho sentido. En la cima de la pirámide laplaciana tenemos la imagen de menor resolución, que coincide con la cima de la pirámide gaussiana, y en los distintos niveles laplacianos almacenamos precisamente la información de detalle que se fue perdiendo.

Por tanto, para reconstruir la imagen podemos partir de la cima de la pirámide laplaciana, expandirla hasta el tamaño del nivel inmediatamente inferior y sumarle el nivel laplaciano correspondiente. Como antes habíamos calculado ese nivel laplaciano mediante una resta, ahora, al sumar, recuperamos exactamente el nivel anterior de la pirámide gaussiana.

Después repetimos el mismo procedimiento: expandimos ese nivel reconstruido, le sumamos el siguiente nivel de la pirámide laplaciana y recuperamos el siguiente nivel de la pirámide gaussiana. Continuamos así hasta llegar a la base.

Al final recuperamos la imagen original. En esencia, es pura aritmética: lo que antes restamos, ahora lo volvemos a sumar.

Gaussian Pyramid



27 of 90

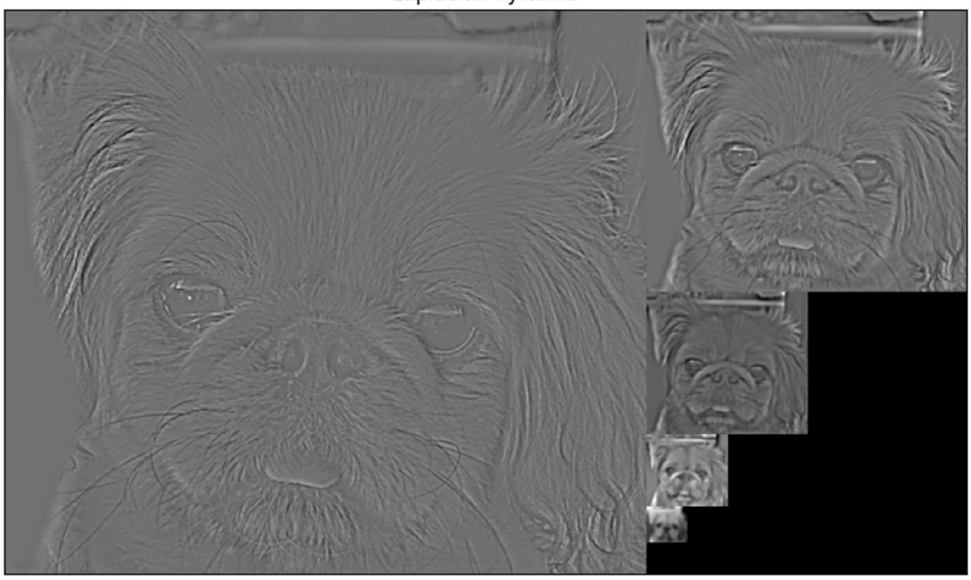
Ahora, en esta diapositiva y en las dos siguientes, vemos una pirámide gaussiana de cuatro niveles construida a partir de la imagen de un pequinés.

A continuación, veremos la pirámide laplaciana correspondiente y, finalmente, compararemos la imagen original con la imagen reconstruida a partir de la pirámide laplaciana.

La reconstrucción es exacta: todos los píxeles de la imagen reconstruida coinciden con los de la imagen original.

Laplacian Pyramid

Laplacian Pyramid



Laplacian Pyramid

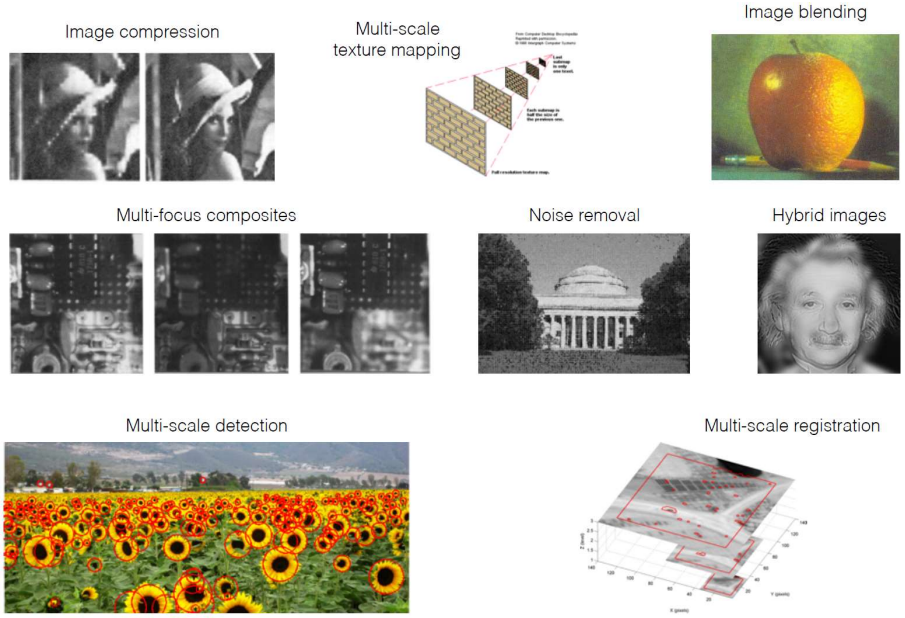
Original Image **Reconstructed Image**



All pixels are the same!! We have a reduced version of the original image, and all the details (high frequencies) at different scales. So, we have all the necessary information for a perfect reconstruction!

29 of 90

What are image pyramids used for?



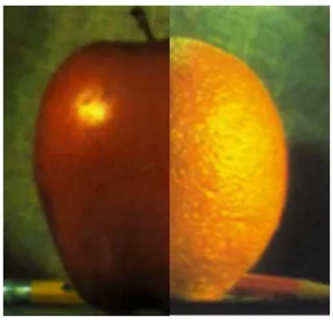
16-385 Computer Vision (Kris Kitani)
Carnegie Mellon University

¿Para qué se utilizan las pirámides de imágenes a nivel práctico? En realidad, aparecen en una gran variedad de tareas.

Se han utilizado en compresión de imágenes, image blending, construcción de imágenes híbridas, detección multiescala y registro multiescala, entre otras aplicaciones.

En definitiva, las pirámides de imágenes han aparecido en multitud de tareas y algoritmos a lo largo de la historia de la visión por computador.

Image Blending



vs



1) **Generate the Laplacian pyramid (L_A and L_B) for both input images (A and B)**
   - To do so, you first need to compute Gaussian pyramids
2) **Build a Gaussian pyramid G_R from the selected region/mask R**
   - R: mask that says which pixels come from image A and which from image B
3) **Create a combined Laplacian pyramid L from L_A and L_B using pixels of G_R as weights**:
   $$L(i,j) = G_R(i,j)L_A(i,j) + \big(1 - G_R(i,j)\big)L_B(i,j)$$
4) **Collapse this new mixed/hybrid Laplacian pyramid (L), and reconstruct the "original" image**

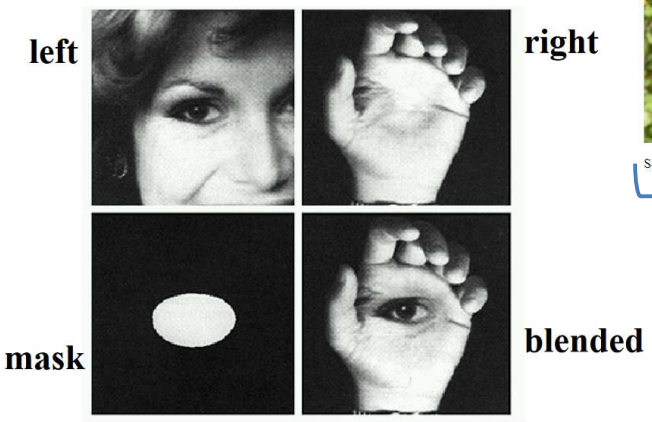
31 of 90

Por ejemplo, si nos centramos en image blending, lo que buscamos es combinar dos imágenes de manera que la transición entre ellas, es decir, la costura, apenas sea perceptible. En esta diapositiva vemos el ejemplo clásico de una manzana y una naranja, que se combinan para obtener una imagen híbrida en la que la zona de unión resulte suave y natural.

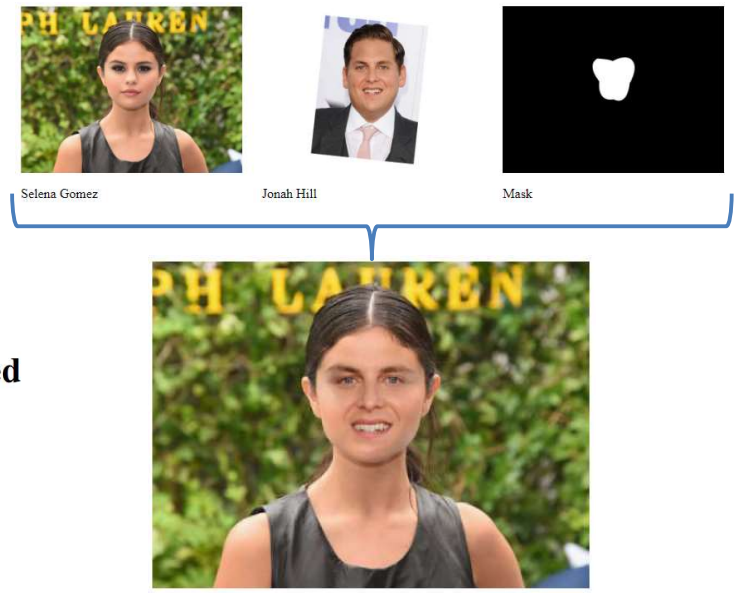
Para ello, si llamamos A y B a las dos imágenes de entrada, el primer paso consiste en construir la pirámide gaussiana y, a partir de ella, la pirámide laplaciana de cada imagen. Además, necesitamos una máscara R, que es una imagen binaria que indica qué regiones deben tomarse de A y cuáles de B. A esa máscara también se le construye una pirámide gaussiana, de manera que la transición entre ambas imágenes se produzca de forma progresiva y no brusca.

A continuación, en cada nivel, se combinan las dos pirámides laplacianas utilizando como pesos los valores de la pirámide gaussiana de la máscara. De este modo obtenemos una pirámide laplaciana híbrida. Finalmente, esa pirámide se reconstruye o se colapsa, igual que vimos en diapositivas anteriores, para obtener la imagen final mezclada. El resultado es una imagen en la que las dos fuentes quedan integradas de forma mucho más natural que si simplemente pegásemos una mitad con la otra.

Image Blending



Burt and Adelson, "A multiresolution spline with application to image mosaics," ACM Transactions on Graphics, 1983, Vol.2, pp.217-236.



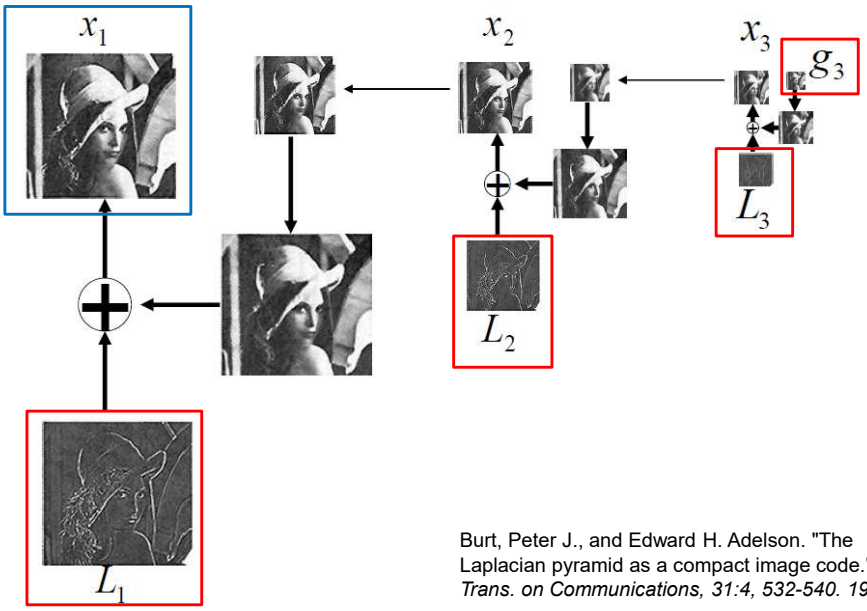
https://inst.eecs.berkeley.edu/~cs194-26/fa17/upload/files/proj3/cs194-26-abq/ 32 of 90

En esta diapositiva podemos ver algunos ejemplos de image blending. A la izquierda observamos la combinación de la imagen de un ojo y una mano, utilizando una máscara con forma aproximadamente ovalada para determinar la región de mezcla.

A la derecha vemos otro ejemplo en el que se combinan la cara de Selena Gomez y la de Jonah Hill, utilizando una máscara adaptada a la región facial. De nuevo, la máscara determina qué zonas proceden de cada una de las imágenes y permite realizar una transición suave entre ambas.

Image Compression

- To reconstruct the original image (x_1), we just need to store L_1, L_2, L_3 and g_3



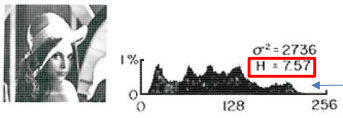
Burt, Peter J., and Edward H. Adelson. "The Laplacian pyramid as a compact image code." *IEEE Trans. on Communications, 31:4, 532-540. 1983.*

Otra aplicación clásica de las pirámides es la compresión de imágenes. Recordemos que, para reconstruir la imagen original, que en esta diapositiva denominamos x1, no necesitamos almacenar todos los niveles de la pirámide gaussiana.

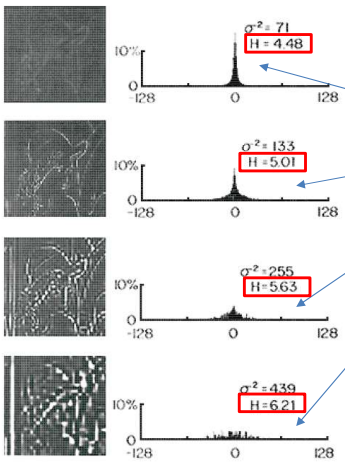
En este caso basta con almacenar L1, L2, L3 y g3. g3 corresponde a la cima de la pirámide gaussiana, que coincide también con la cima de la pirámide laplaciana, mientras que L1, L2 y L3 representan los distintos niveles de detalle de la pirámide laplaciana.

A partir de esta información podemos reconstruir sucesivamente los niveles de la pirámide gaussiana hasta recuperar la imagen original.

Image Compression



Histogram of the original image



Distribution of pixel gray level values at various stages of the encoding process.

Pixel values in the Laplacian pyramid are concentrated near zero

Histogram entropy represents the minimum number of bits per pixel required to exactly encode the image. The maximum entropy would be 8, since the image is initially represented at 256 gray levels, and would be obtained when all gray levels were equally likely.

Burt, Peter J., and Edward H. Adelson. "The Laplacian pyramid as a compact image code." *IEEE Trans. on Communications, 31:4, 532-540. 1983.*

Esta idea de compresión se entiende mejor si observamos los histogramas. En la imagen original encontramos una gran diversidad de niveles de gris, desde valores próximos a 0 hasta valores próximos a 255. Por tanto, existe una elevada variabilidad y la entropía del histograma es alta.

La entropía nos proporciona, en términos ideales, un límite inferior para el número medio de bits por píxel necesario para codificar sin pérdida los valores de la imagen. En una imagen de 256 niveles de gris, la entropía máxima es de 8 bits por píxel, y se alcanza cuando los 256 niveles son igualmente probables. En este ejemplo, la imagen original tiene una entropía de 7,57.

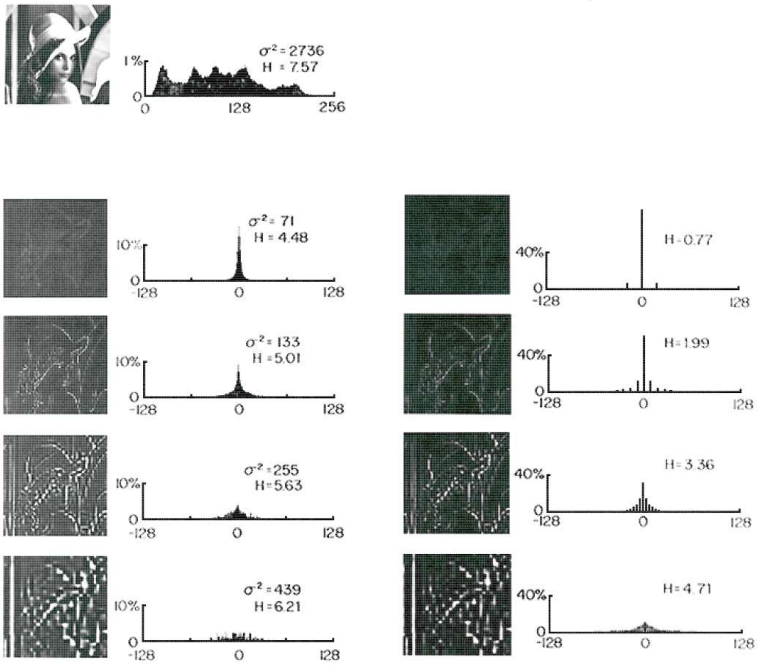
Cuando pasamos a los niveles de la pirámide laplaciana, la situación cambia. Como estamos almacenando fundamentalmente información de detalle, es decir, diferencias entre escalas, podemos obtener tanto valores positivos como negativos. Además, la mayor parte de los coeficientes suelen concentrarse alrededor de 0, porque en muchas regiones la diferencia entre un nivel de la pirámide gaussiana y la predicción obtenida a partir del nivel superior es muy pequeña.

Por tanto, los histogramas de los niveles laplacianos están mucho más concentrados alrededor de 0. En esta diapositiva, por ejemplo, observamos entropías de 4,48, 5,01, 5,63 y 6,21 para distintos niveles.

Además, debemos tener en cuenta que, aunque la entropía pueda aumentar en algunos de los niveles superiores, estamos trabajando con imágenes cada vez más pequeñas. Por tanto, el número total de valores que tenemos que almacenar

también disminuye considerablemente.

Image Compression



Substantial further reduction is obtained through quantization

4.48 vs 0.77

5.01 vs 1.99

5.63 vs 3.36

6.21 vs 4.71

Burt, Peter J., and Edward H. Adelson. "The Laplacian pyramid as a compact image code." *IEEE Trans. on Communications, 31:4, 532-540. 1983.*

Estos histogramas correspondientes a los niveles de la pirámide laplaciana pueden simplificarse todavía más mediante cuantización.

La idea consiste en representar conjuntos de valores próximos mediante un mismo valor cuantizado, reduciendo así el número de valores distintos que tenemos que codificar. Como consecuencia, disminuye también la entropía.

Por ejemplo, en esta diapositiva vemos cómo una entropía de 4,48 puede reducirse hasta 0,77; una de 5,01 hasta 1,99; una de 5,63 hasta 3,36; y una de 6,21 hasta 4,71.

Evidentemente, aquí ya estamos introduciendo pérdida de información: la cuantización permite reducir considerablemente el número de bits necesarios, pero la reconstrucción deja de ser exactamente idéntica a la imagen original.

Image Compression

ORIGINAL **COMPRESSED**



(a)



(b)

1.58 bits/pixel

The encoded images are almost indistinguishable from the originals



(c)



(d)

0.73 bits/pixel

Burt, Peter J., and Edward H. Adelson. "The Laplacian pyramid as a compact image code." *IEEE Trans. on Communications, 31:4, 532-540. 1983.*

Y aquí es donde alcanzaríamos una tasa de codificación de, por ejemplo, 1,58 bits por píxel. Esto significa que, en promedio, cada píxel de la imagen original requiere 1,58 bits de información para ser codificado.

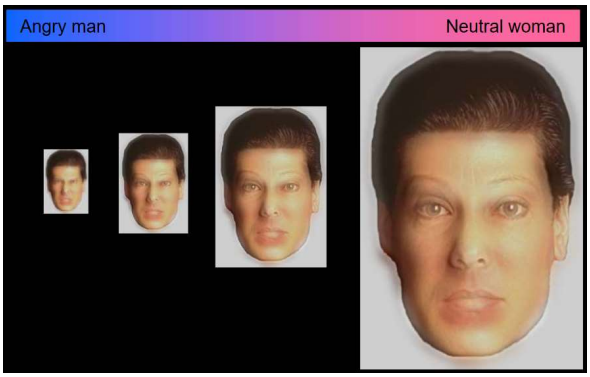
En lugar de necesitar 7,57 bits por píxel, como en la imagen original, pasamos a necesitar aproximadamente 1,58 bits por píxel para toda la pirámide Laplaciana cuantizada. Esto equivale a alrededor del 20,87 % del tamaño original, es decir, una reducción de aproximadamente el 79 %.

Estamos, por tanto, ante una compresión con pérdida, pero en este ejemplo las imágenes reconstruidas son visualmente prácticamente indistinguibles de las originales.

Hybrid Images

A. Oliva, A. Torralba, P.G. Schyns (2006). Hybrid Images. ACM Transactions on Graphics. http://olivalab.mit.edu/hybridimage.htm

- By properly **mixing** a part of the **high frequencies of one image with** a part of the **low frequencies of another image,** we obtain a **hybrid image** that **allows different interpretations at different distances.**



We evaluate the final result using the Gaussian Pyramid!!

Low frequencies prevail at **long distances**, while **high frequencies** prevail at **short distances**.

37 of 90

Y finalmente tenemos el ejemplo de las imágenes híbridas. La idea consiste en combinar adecuadamente la información de altas frecuencias de una imagen con la información de bajas frecuencias de otra.

De este modo obtenemos una imagen híbrida, que puede dar lugar a interpretaciones diferentes dependiendo de la distancia de observación.

Por ejemplo, en esta diapositiva, cuando observamos la imagen de cerca, a la derecha, percibimos principalmente la cara neutra de una mujer. En cambio, al alejarnos, hacia la izquierda, pasa a dominar la imagen de un hombre enfadado.

La idea clave es que las altas frecuencias predominan a corta distancia, mientras que las bajas frecuencias predominan a larga distancia.

Este efecto puede analizarse precisamente utilizando una pirámide gaussiana, porque al avanzar hacia niveles más suavizados vamos eliminando progresivamente las altas frecuencias y observamos qué componente de la imagen sigue siendo perceptible.

Thinking in Frequency

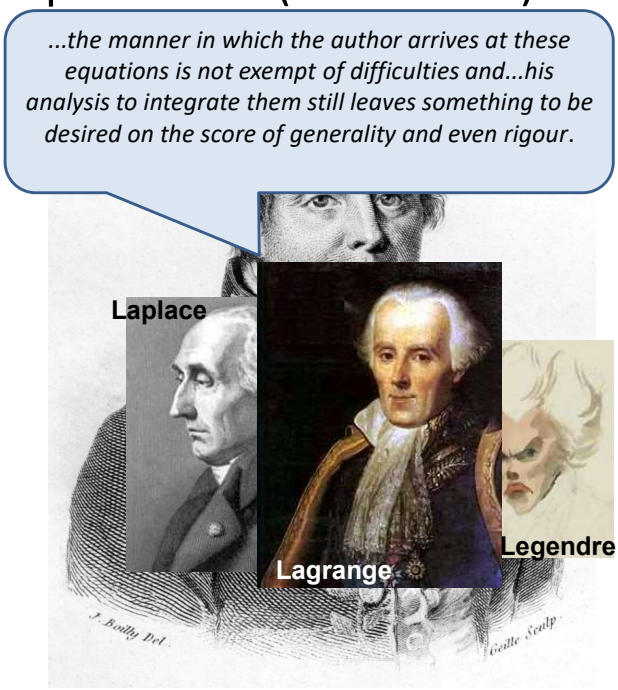
Ahora vamos a adentrarnos en el dominio de la frecuencia.

Jean-Baptiste Joseph Fourier (1768-1830)

had crazy idea (1807):

"Any" function can be rewritten as a weighted sum of sines and cosines of different frequencies.

- Don't believe it?
  - Neither did Lagrange, Laplace, Poisson and others
  - Not translated into English until 1878!
- Now, this is called Fourier Series.



Slide: Efros

Un poco el protagonista de esta historia es Jean-Baptiste Joseph Fourier, que tuvo una idea bastante revolucionaria a comienzos del siglo XIX. Propuso que una función periódica, bajo determinadas condiciones, podía representarse como una suma ponderada de senos y cosenos de distintas frecuencias.

Si esta idea cuesta creerla, tampoco debemos extrañarnos demasiado, porque en su momento tampoco convenció inmediatamente a matemáticos como Lagrange, Laplace o Poisson. De hecho, el trabajo de Fourier fue inicialmente recibido con bastante escepticismo.

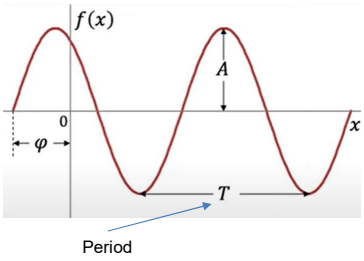
Con el tiempo, estas ideas dieron lugar a lo que hoy conocemos como series de Fourier y, posteriormente, a toda la teoría asociada a la transformada de Fourier.

A sum of sinusoids

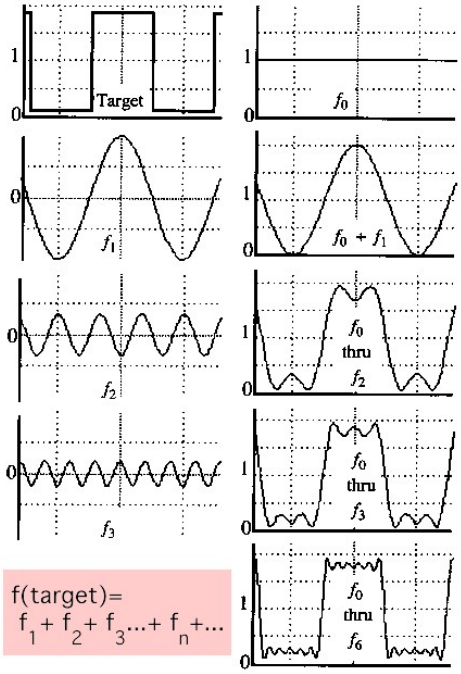
Our building block:

$$f(x) = Asin(2\pi ux + \varphi)$$

Amplitude Frequency (1/T) Phase



Add enough of them to get any signal *s(x)* you want!



f(target)= $f_1 + f_2 + f_3 ... + f_n + ...$

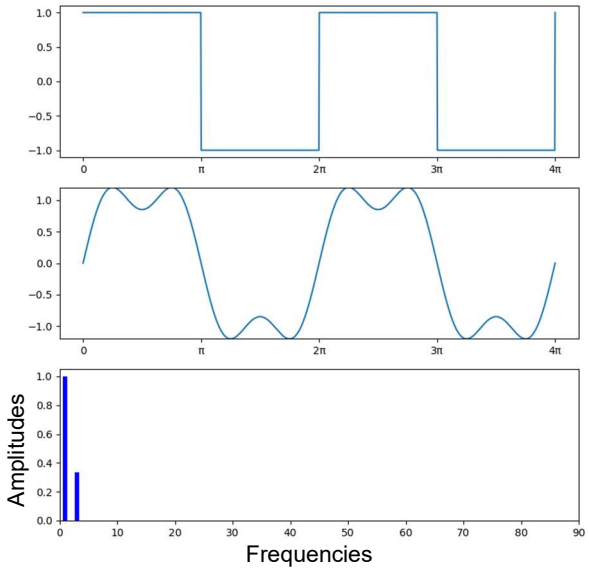
40 of 90

En esta diapositiva vemos de manera visual la idea central de las series de Fourier. Nuestro bloque básico es una sinusoide, que queda determinada por tres parámetros fundamentales: su amplitud, su frecuencia y su fase.

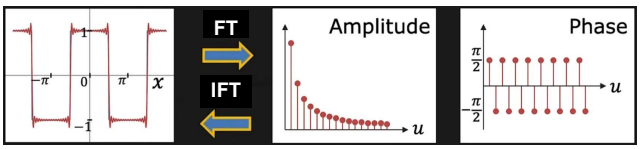
A partir de ahí, la idea consiste en sumar sinusoides de distintas frecuencias, amplitudes y fases. En el ejemplo de la derecha partimos de una señal objetivo con forma aproximadamente rectangular. Si utilizamos únicamente una componente sinusoidal, la aproximación es muy pobre. Al añadir una segunda componente, una tercera, una cuarta, etcétera, la suma se va pareciendo cada vez más a la señal objetivo.

Por tanto, la idea esencial es que una señal puede descomponerse como combinación de señales sinusoidales más simples. Cuantas más componentes incorporemos, mejor podremos aproximar la señal.

A sum of sinusoids



Frequency representation of signal



The **Fourier Transform (FT)** represents a signal $f(x)$ in terms of Amplitudes and Phases of its constituent sinusoids.

The **Inverse Fourier Transform (IFT)** computes the signal $f(x)$ from the Amplitudes and Phases of its constituent sinusoids.

No loss of information!!!

Slide adapted from Derek Hoiem & Shree Nayar

En esta diapositiva damos un paso más. A la izquierda arriba tenemos una señal periódica, y debajo vemos cómo podemos aproximarla mediante la suma de un número creciente de componentes sinusoidales. Al incorporar más frecuencias, la reconstrucción se parece cada vez más a la señal original (aunque alrededor de las discontinuidades aparecen unas oscilaciones características).

La gráfica inferior nos muestra precisamente otra forma de representar esa misma señal: en lugar de describir cómo varía su valor con x, representamos qué frecuencias están presentes y con qué amplitud contribuye cada una. Es decir, pasamos del dominio espacial, o en una señal 1D del dominio de la variable x, al dominio de la frecuencia.

Eso es, esencialmente, lo que hace la transformada de Fourier. A partir de la señal f(x), obtenemos una representación en frecuencia que nos indica la amplitud y la fase de las distintas componentes sinusoidales que la constituyen.

Y el proceso es reversible. Mediante la transformada inversa de Fourier podemos partir de esas amplitudes y fases y reconstruir nuevamente la señal original. Por eso aparece abajo la idea de “No loss of information”: pasar al dominio de Fourier no implica, por sí mismo, perder información. Estamos simplemente representando exactamente la misma señal de otra manera.

Fourier Transform is Complex

$f(x)$ → **Fourier Transform** → $F(u)$

$$f(x) = \int_{-\infty}^{\infty} F(u) e^{j2\pi ux} du \qquad F(u) = \int_{-\infty}^{\infty} f(x) e^{-j2\pi u} \, dx$$

For every frequency u, $F(u)$ holds magnitude (A) and phase (φ) of the corresponding sinusoid.

How can we do it? For mathematical convenience, we use **complex numbers**!

The **magnitude** tells "how much" of a certain frequency component is present.

The **phase** tells "where" the frequency component is in the signal/image.

Note: We could say that **amplitude** is the peak value of a sinusoid in the **time/spatial domain**, while Fourier **magnitude** indicates the strength of a given frequency component in the **Fourier domain**.

Rectangular or Cartesian form

$$F(u) = R(u) + jI(u)$$

$$A = |F(u)| = \sqrt{R(u)^2 + I(u)^2} \qquad \varphi = tan^{-1} \frac{I(u)}{R(u)}$$

⟺

Polar form

$$F(u) = Ae^{j\varphi}$$
$$= A(cos\varphi + jsin\varphi)$$

Complex Exponential (Euler's Formula)

42 of 90

En esta diapositiva introducimos una idea importante: la transformada de Fourier es una función compleja. Es decir, para cada frecuencia u, la transformada F(u) nos proporciona un número complejo que contiene dos tipos de información: una magnitud y una fase.

La magnitud nos indica cuánto contribuye esa frecuencia concreta a la señal. Dicho de otra manera, nos indica el peso o importancia de esa componente frecuencial. La fase, en cambio, nos proporciona información sobre dónde se encuentra esa componente dentro de la señal.

Para manejar estos valores utilizamos números complejos. Podemos representar F(u) en forma cartesiana, mediante una parte real R(u) y una parte imaginaria I(u), o de manera equivalente en forma polar, mediante una magnitud A y una fase phi.

La magnitud se obtiene a partir de las partes real e imaginaria y la fase representa el ángulo del número complejo. Ambas representaciones contienen exactamente la misma información y podemos pasar de una a otra.

Convolution Theorem

- The Fourier Transform of the convolution of two functions is equal to the pointwise multiplication of their Fourier Transforms.

$$FT(g * h) = FT(g) \cdot FT(h)$$

- **Convolution in spatial domain is equivalent to multiplication in frequency domain!**

- The symmetric/dual property is also true:

$$FT(g \cdot h) = FT(g) * FT(h)$$

En esta diapositiva introducimos una propiedad fundamental de la transformada de Fourier: el teorema de la convolución.

Lo que nos dice es que convolucionar dos funciones en el dominio espacial equivale a multiplicar punto a punto sus transformadas de Fourier en el dominio de la frecuencia.

Dicho de otra manera, si tenemos dos funciones g y h, podemos seguir dos caminos equivalentes. Primer camino: convolucionamos g y h en el dominio espacial y, después, calculamos la transformada de Fourier del resultado. Segundo camino: calculamos por separado la transformada de Fourier de g y de h y, después, multiplicamos ambas transformadas punto a punto. Ambos caminos producen exactamente el mismo resultado en el dominio de la frecuencia.

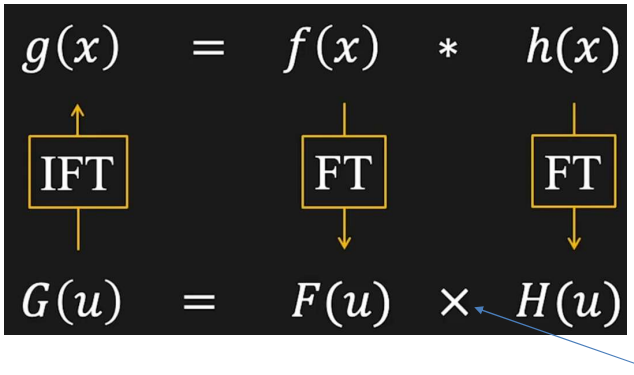
El teorema de la convolución nos dice que convolucionar dos funciones en el dominio espacial y después transformar el resultado a Fourier es equivalente a transformar primero cada función, multiplicar sus transformadas punto a punto y, si queremos volver al dominio espacial, aplicar finalmente la transformada inversa.

Esta propiedad es especialmente importante porque nos permite interpretar qué hace un filtro desde el punto de vista frecuencial. Cuando convolucionamos una imagen con un filtro paso bajo, por ejemplo, en frecuencia lo que estamos haciendo es multiplicar el espectro de la imagen por una función que atenúa las altas frecuencias.

La propiedad dual también es cierta: multiplicar dos funciones en el dominio

espacial corresponde a convolucionar sus transformadas en el dominio de la frecuencia.

Convolution using Fourier Transform



Why is all this useful?

1) It can be faster (particularly for large filters)!

   See Heckbert's and Kundur's notes regarding *Fast Fourier Transform*.
   See, for instance, cv2.filter2D documentation: "The function uses the DFT-based algorithm in case of sufficiently large kernels (~11x11 or larger) and the direct algorithm for small kernels."

2) This perspective allows us to interpret image filtering in terms of frequencies!

Slide adapted from Shree Nayar

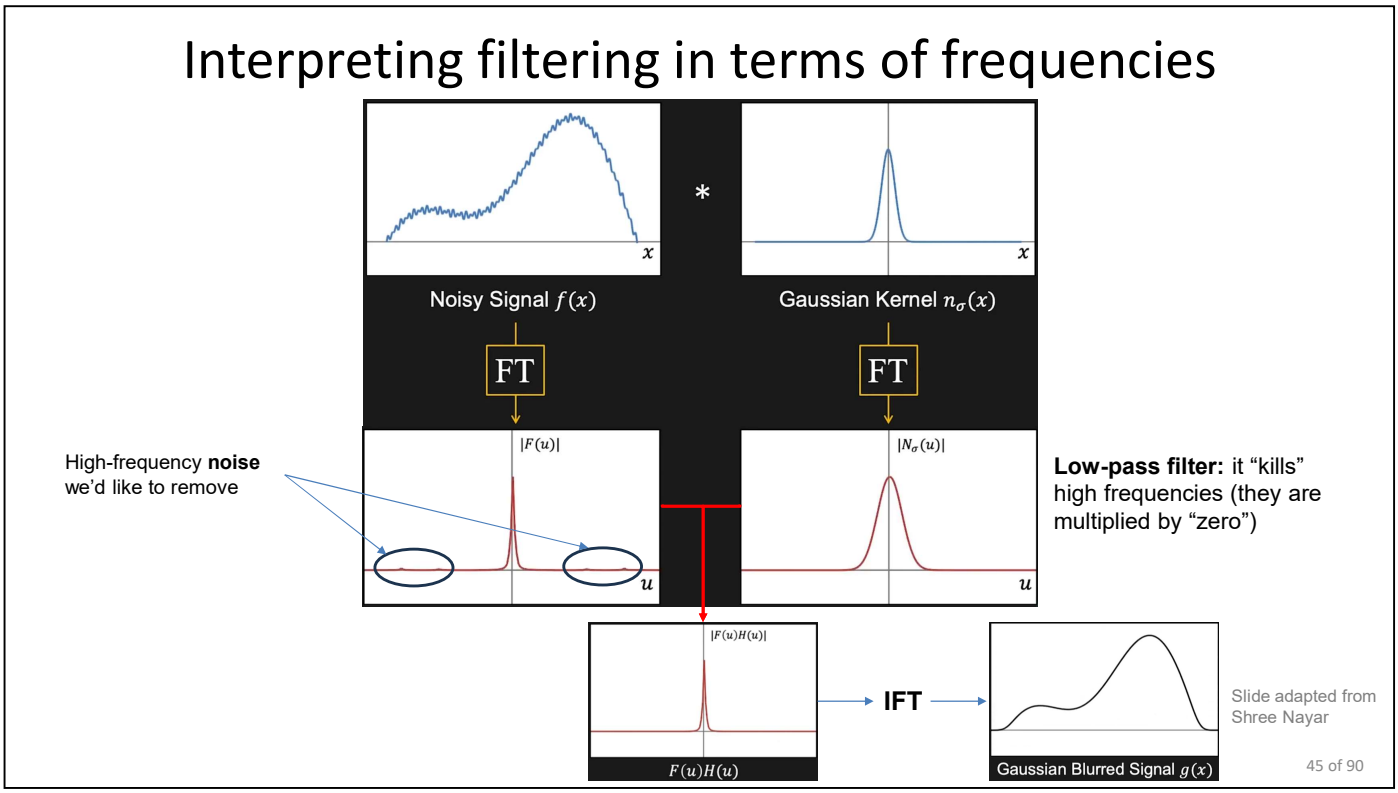
En esta diapositiva vemos de manera explícita cómo podemos realizar una convolución utilizando la transformada de Fourier.

Partimos de dos funciones, f(x) y h(x), cuya convolución produce g(x). En lugar de calcular directamente esa convolución en el dominio espacial, podemos calcular la transformada de Fourier de f y de h, obteniendo F(u) y H(u). A continuación, multiplicamos ambas transformadas punto a punto. El resultado de esa multiplicación es G(u), es decir, la transformada de Fourier de la señal resultante g(x). Finalmente, aplicamos la transformada inversa de Fourier a G(u) y recuperamos g(x).

Por tanto, tenemos dos caminos equivalentes: convolucionar directamente en el dominio espacial, o transformar ambas funciones al dominio de la frecuencia, multiplicarlas punto a punto y aplicar después la transformada inversa.

¿Por qué puede resultar útil hacer todo esto? En primer lugar, porque para kernels suficientemente grandes puede ser computacionalmente más eficiente realizar el filtrado mediante la transformada rápida de Fourier que aplicar directamente el kernel posición por posición. De hecho, OpenCV utiliza un algoritmo basado en DFT (Discrete Fourier Transform) para kernels suficientemente grandes, mientras que para kernels pequeños utiliza el algoritmo directo.

Y, en segundo lugar, porque esta perspectiva nos permite interpretar el filtrado en términos de frecuencias. Podemos observar directamente qué componentes frecuenciales de la imagen son atenuadas, preservadas o amplificadas por el filtro.



En esta diapositiva vemos de forma muy clara cómo interpretar un filtrado en términos de frecuencia.

Arriba a la izquierda tenemos una señal ruidosa f(x). Ese ruido contiene componentes de alta frecuencia que queremos eliminar. Arriba a la derecha tenemos un kernel gaussiano, que utilizamos como filtro de suavizado.

Si calculamos la transformada de Fourier de la señal, obtenemos su espectro F(u). En ese espectro podemos identificar las bajas frecuencias cerca del origen y, más alejadas, las componentes de alta frecuencia que corresponden al ruido.

Si calculamos ahora la transformada de Fourier del kernel gaussiano, obtenemos otra gaussiana en el dominio de la frecuencia. Esta transformada tiene valores altos en torno a frecuencia cero y decrece progresivamente a medida que aumenta la frecuencia. Por eso decimos que la gaussiana actúa como filtro paso bajo.

Aplicando el teorema de la convolución, convolucionar la señal con el kernel gaussiano en el dominio espacial equivale a multiplicar punto a punto F(u) por N(u) en el dominio de la frecuencia. Como N(u) tiene valores altos en bajas frecuencias y valores muy pequeños en altas frecuencias, las componentes de baja frecuencia se conservan mientras que las de alta frecuencia quedan fuertemente atenuadas.

Finalmente aplicamos la transformada inversa de Fourier y obtenemos la señal suavizada en el dominio espacial.

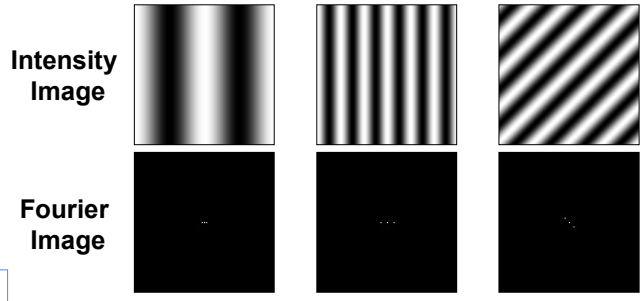
Fourier Analysis in Images

- According to Fourier, signals can be decomposed into sums of sines and cosines. But... can this be applied to images? 🤔
- Of course! An image is a visual signal!



"Any" signal can be expressed as a sum of a series of sinusoids

Note: What we usually display as a Fourier image is the magnitude. The Fourier transform itself is complex and also contains phase.



In our case, these are sinusoidal variations in brightness across the image (i.e. image information is represented not for each pixel separately but rather for each frequency)

46 of 90

En esta diapositiva damos el salto desde señales unidimensionales a imágenes. La idea de Fourier se aplica igualmente porque una imagen también puede entenderse como una señal, en este caso bidimensional.

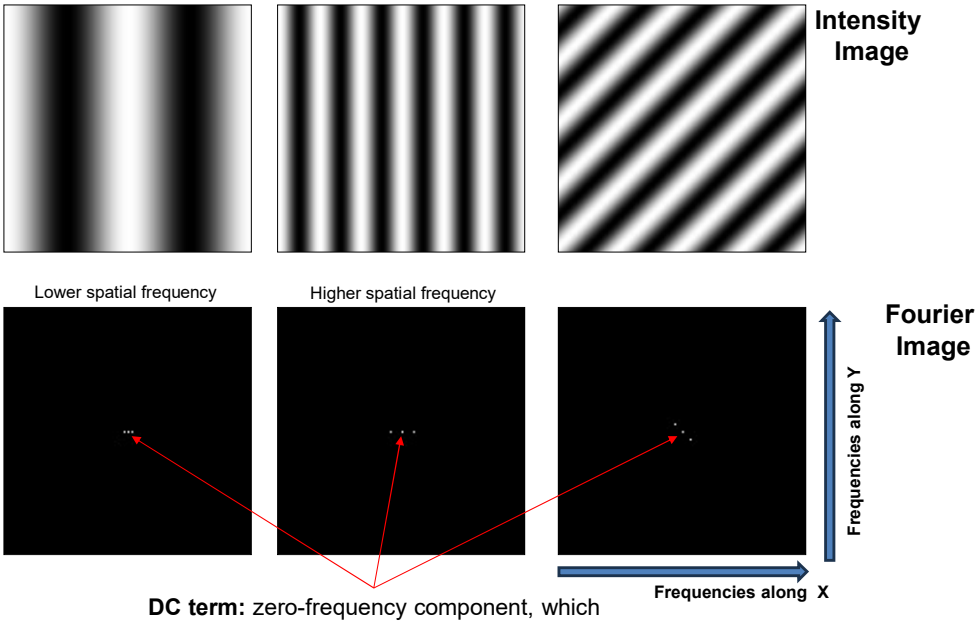
En lugar de descomponer una señal 1D en sinusoides de distintas frecuencias, ahora podemos pensar una imagen como una suma de patrones sinusoidales bidimensionales. Esos patrones pueden variar en frecuencia y también en orientación.

Los ejemplos de la derecha muestran precisamente eso. Arriba tenemos patrones sinusoidales de intensidad: uno con variación relativamente lenta, otro con una frecuencia espacial mayor y otro orientado en diagonal. Debajo aparece la representación de Fourier correspondiente.

En esas imágenes de Fourier, cada patrón sinusoidal produce unos pocos puntos muy localizados en el dominio de la frecuencia. La posición de esos puntos nos informa de la frecuencia espacial y de la orientación del patrón, mientras que su intensidad representa la magnitud de esa componente.

En esta diapositiva estamos mostrando únicamente la magnitud de la transformada de Fourier. La fase también existe y contiene información esencial, pero no se visualiza aquí.

Fourier Analysis in Images



Two bright pixels, either side of the center, encode the (single) sinusoidal pattern (i.e. frequencies +*k* and –*k*).

Every pixel of the Fourier image is a spatial frequency value. The magnitude of that value (i.e. amplitude/contrast of a constituent sinusoid) **is encoded by the brightness of the pixel.**

Since there is only one sinusoidal frequency component in each image, the Fourier magnitude contains only one symmetric pair of non-zero components (together with the DC term); all other values are zero.

DC term: zero-frequency component, which encodes the average brightness of the whole image.

47 of 90

En esta diapositiva profundizamos un poco más en cómo interpretar la transformada de Fourier de una imagen.

Arriba tenemos tres patrones sinusoidales bidimensionales. El primero tiene una frecuencia espacial relativamente baja: las bandas son anchas y cambian lentamente. El segundo tiene una frecuencia espacial mayor: las bandas son más estrechas y se repiten más veces. El tercero tiene una orientación diagonal.

Debajo vemos la magnitud de la transformada de Fourier correspondiente. Para una imagen sinusoidal pura aparecen dos puntos brillantes simétricos respecto al centro. Esos dos puntos representan las dos frecuencias opuestas, +f y -f, asociadas a una misma sinusoide real. Esta duplicidad aparece porque una sinusoide real puede expresarse como la suma de dos exponenciales complejas de frecuencias opuestas.

La distancia al centro nos indica la frecuencia espacial. Cuanto más alejados del centro estén esos puntos, mayor es la frecuencia. Por eso, en la segunda imagen, que tiene bandas más finas, los puntos aparecen más alejados del centro que en la primera.

La dirección en la que aparecen esos puntos también codifica la orientación de la variación espacial. En el caso diagonal, los puntos aparecen igualmente desplazados en una dirección diagonal en el dominio de Fourier.

El centro de la imagen de Fourier corresponde a la componente DC, es decir, a frecuencia cero. Esa componente codifica el valor medio de intensidad de la

imagen.

Cada píxel de la imagen de Fourier corresponde, por tanto, a una determinada frecuencia espacial en x y en y. Su magnitud nos indica cuánto contribuye esa frecuencia a la imagen.

Fourier Analysis in Images



Slide: Derek Hoiem

48 of 90

En este GIF podemos observar cómo cambia la representación de Fourier a medida que aumenta la frecuencia espacial de las franjas. Los dos puntos simétricos se van alejando progresivamente del centro, indicando una frecuencia espacial cada vez mayor.

La componente DC, que corresponde al valor medio de intensidad de la imagen y aparecería en el centro del espectro, no se muestra en esta visualización. Puede haberse omitido deliberadamente para simplificar el ejemplo y centrar la atención en el par de componentes asociado a la frecuencia de la sinusoide.

Otra posibilidad es que la señal sinusoidal tenga media cero o aproximadamente cero. Aunque visualmente se muestre como una imagen en escala de grises, sus valores reales podrían oscilar, por ejemplo, entre -100 y +100. En ese caso, la componente DC sería nula o prácticamente nula.

Fourier Analysis in Images

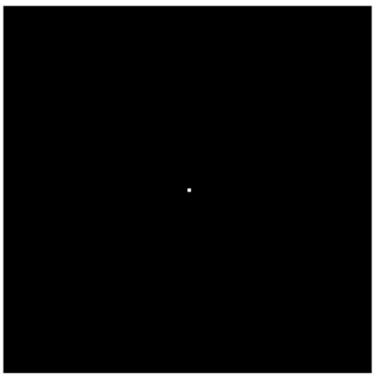


Slide: Derek Hoiem

Lo mismo ocurre en esta diapositiva, pero ahora con un patrón sinusoidal cuya intensidad varía a lo largo de la dirección horizontal. A medida que aumenta la frecuencia espacial, el par de puntos simétricos de la transformada de Fourier se aleja progresivamente del centro, en este caso a lo largo del eje horizontal de frecuencias.

Visualmente, este patrón está formado por franjas verticales, precisamente porque la intensidad cambia al desplazarnos horizontalmente. La dirección en la que aparecen las componentes en Fourier corresponde a la dirección de variación de la intensidad y, por tanto, es perpendicular a la orientación de las franjas.

Fourier Analysis in Images





Slide: Derek Hoiem

Fourier Analysis in Images



Slide: Derek Hoiem

51 of 90

En esta diapositiva vemos un ejemplo algo más complejo que los anteriores. La imagen de la derecha ya no contiene una única variación sinusoidal, sino la combinación de varias componentes frecuenciales.

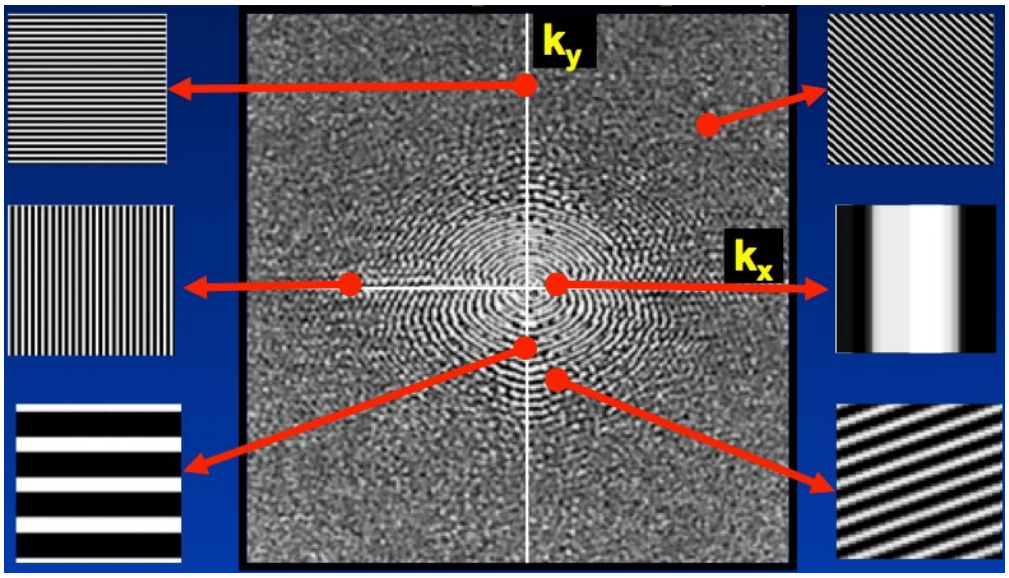
Eso se refleja directamente en su transformada de Fourier, que aparece a la izquierda. En lugar de observar un único par de puntos simétricos, vemos varios pares de componentes. Cada par corresponde a una determinada frecuencia espacial y a una determinada dirección de variación presente en la imagen.

La idea importante es que una textura aparentemente compleja puede entenderse como la superposición de varias estructuras sinusoidales más simples. En el dominio de Fourier, esas componentes quedan separadas y aparecen como picos en posiciones diferentes.

Además, la distancia de cada pico respecto al centro nos informa de su frecuencia espacial: cuanto más alejado esté, mayor es la frecuencia. La dirección respecto al centro nos informa de la orientación de la variación correspondiente.

En este caso, como la imagen combina varias orientaciones o periodicidades, aparecen varios pares simétricos de puntos en el espectro.

Fourier Analysis in Images



It's symmetrical about the origin:
1st = 3rd quadrant
2nd = 4th quadrant

Slide: Karla Miller

Each point in the Fourier Image corresponds to a particular spatial frequency

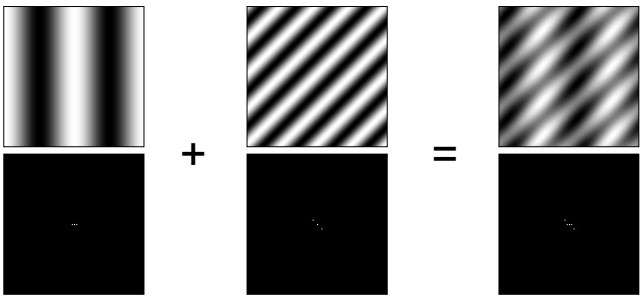
52 of 90

En esta diapositiva resumimos cómo interpretar geométricamente la transformada de Fourier de una imagen bidimensional. Cada punto del dominio de Fourier representa una componente de frecuencia espacial caracterizada por dos valores: kx, que indica la frecuencia en la dirección horizontal, y ky, que indica la frecuencia en la dirección vertical.

La distancia de un punto al centro nos informa de la frecuencia espacial: cuanto más lejos del centro, mayor es la frecuencia. Su dirección respecto al centro nos informa de la dirección en la que varía la intensidad de la imagen. Por eso, la orientación de los puntos en Fourier es perpendicular a la orientación visual de las franjas. Por ejemplo, unas franjas horizontales varían verticalmente y generan componentes sobre el eje vertical; unas franjas verticales varían horizontalmente y generan componentes sobre el eje horizontal.

En una imagen real, si visualizamos únicamente la magnitud, obtenemos una imagen simétrica respecto al centro: lo que aparece en el primer cuadrante tiene su correspondiente componente en el tercero, y lo mismo ocurre entre el segundo y el cuarto cuadrante.

Signals can be composed



The Fourier representation allows manipulation of the global information content of the image by spatial manipulations of the transformed image.

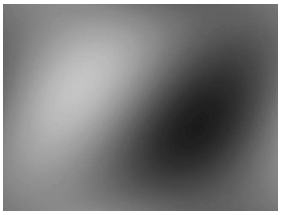
Original



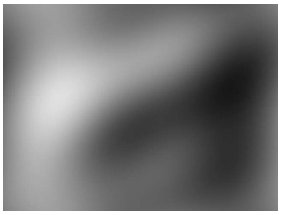
First Frequency (Average)



Second Frequency



Third Frequency



Using 50% of frequencies



https://photo.stackexchange.com/questions/40401/what-does-frequency-mean-in-an-image

53 of 90

En esta diapositiva reforzamos la idea de que una señal puede construirse como suma de componentes sinusoidales simples. En la parte superior vemos dos patrones sinusoidales diferentes y cómo, al sumarlos, obtenemos una señal más compleja. En el dominio de Fourier, esa suma se traduce en la presencia simultánea de las componentes frecuenciales correspondientes a ambos patrones.

En la parte inferior vemos una reconstrucción progresiva de la imagen a partir de sus componentes de Fourier. La primera reconstrucción conserva únicamente la componente DC, es decir, el valor medio de intensidad de la imagen. A continuación, se van incorporando progresivamente nuevas componentes frecuenciales. Al principio, al conservar únicamente las frecuencias más bajas, recuperamos fundamentalmente las grandes estructuras y las variaciones suaves de intensidad. Conforme incorporamos frecuencias cada vez más altas, vamos recuperando detalles más finos de la imagen. Finalmente, cuando conservamos una fracción suficientemente grande del espectro, la imagen comienza a parecerse claramente a la original.

Fourier Analysis in Images (technical note)

```
frequency_x = 0                # Frequency in X-axis
frequency_y = 0.25             # Frequency in Y-axis
amplitude = 100.0              # Sinusoid amplitude
phase = np.pi/4                # Sinusoid phase

# We generate coordinates
x = np.linspace(-10, 10, 100)
y = np.linspace(-10, 10, 100)
x, y = np.meshgrid(x, y)

# We generate the 2D sinusoid
sinusoid_2D = amplitude * np.sin(2 * np.pi * (frequency_x * x + frequency_y * y) + phase)

# Compute the 2-dimensional discrete Fourier Transform.
FT = np.fft.fft2(sinusoid_2D) #https://numpy.org/doc/stable/reference/generated/numpy.fft.fft2.html
# Shift the zero-frequency component to the center of the spectrum.
shifted_FT = np.fft.fftshift(FT) #https://numpy.org/doc/stable/reference/generated/numpy.fft.fftshift.html
# Compute magnitude
magnitude = np.abs(shifted_FT) #sometimes, for visualization purposes, it's convenient to do np.log(np.abs(shiftedFT))

# Display
plt.figure(figsize=(10, 5))
plt.subplot(121), plt.imshow(sinusoid_2D, cmap='gray')
plt.title('Sinusoid'), plt.xticks([]), plt.yticks([])
plt.subplot(122), plt.imshow(magnitude, cmap='gray')
plt.title('Fourier Image'), plt.xticks([]), plt.yticks([])
plt.show()
```


En esta diapositiva mostramos de forma explícita, mediante un pequeño ejemplo en Python, cómo se construye un patrón sinusoidal bidimensional y cómo aparece representado en el dominio de Fourier.

Primero definimos las frecuencias espaciales en las direcciones x e y, además de la amplitud y la fase de la sinusoide. En este ejemplo, frequency_x vale 0 y frequency_y vale 0,25. Esto significa que no existe variación de intensidad en la dirección x y que toda la variación se produce a lo largo de y. Como consecuencia, visualmente obtenemos franjas horizontales.

A continuación, generamos una rejilla bidimensional de coordenadas mediante meshgrid y evaluamos sobre ella la sinusoide. La expresión "amplitude * np.sin(2 * np.pi * (frequency_x * x + frequency_y * y) + phase)" determina el patrón.

Después calculamos la transformada discreta de Fourier bidimensional mediante np.fft.fft2. Como por defecto la frecuencia cero aparece en una esquina, utilizamos fftshift para trasladarla al centro de la representación. Finalmente, calculamos la magnitud mediante el valor absoluto de los coeficientes complejos.

Para esta sinusoide esperamos observar fundamentalmente dos picos simétricos en el dominio de Fourier, correspondientes a las frecuencias espaciales positiva y negativa asociadas a la misma sinusoide real. Como la variación se produce en la dirección vertical, esos dos picos aparecen sobre el eje vertical del espectro.

La fase pi/4 modifica la fase de los coeficientes de Fourier, pero no la posición de los picos ni su magnitud. Por eso, si únicamente visualizamos la magnitud, cambiar

la fase de la sinusoide no debería cambiar esencialmente la imagen de Fourier.

En este ejemplo la señal sinusoidal tiene media prácticamente cero. Aunque visualmente Matplotlib la muestra como una imagen en escala de grises, los valores reales de la señal no están entre 0 y 255. La sinusoide que hemos generado oscila aproximadamente entre -100 y +100. Matplotlib simplemente normaliza esos valores para visualizarlos: los valores menores aparecen oscuros y los mayores claros. Por tanto, el valor medio de la señal es muy próximo a 0 y la componente DC, que representa precisamente ese valor medio, tiene una magnitud muy pequeña. De ahí que en el centro del espectro no aparezca un punto brillante apreciable. Los dos puntos claramente visibles corresponden a las componentes +f y -f de la sinusoide.

Fourier Analysis in Images (technical note)

- Filtering with Fast Fourier Transform (FFT)

```python
import matplotlib.pyplot as plt
import numpy as np

def filter_image(im, fil):
    '''
    im: H x W floating point numpy ndarray representing image in grayscale
    fil: M x M floating point numpy ndarray representing 2D filter
    '''
    H, W = im.shape
    M = fil.shape[0]
    hs = M // 2                              # half of filter size
    fft_shape = (H + M - 1, W + M - 1)       # Minimum size required to compute the full linear convolution
    im_fft = np.fft.fft2(im, s=fft_shape)    # 1) fft im with padding
    fil_fft = np.fft.fft2(fil, s=fft_shape) # 2) fft filter with the same zero-padded FFT size
    im_fil_fft = im_fft * fil_fft            # 3) multiply fft images
    im_fil = np.fft.ifft2(im_fil_fft)        # 4) inverse fft
    im_fil = im_fil[hs:hs + H, hs:hs + W]    # 5) crop to original image size
    im_fil = np.real(im_fil)                 # 6) extract out real part
    return im_fil
```


En esta diapositiva mostramos una nota técnica sobre cómo implementar el filtrado de una imagen mediante FFT aprovechando el teorema de la convolución.

Partimos de una imagen de tamaño H x W y de un filtro cuadrado de tamaño M x M. Para que la multiplicación en el dominio de Fourier corresponda a una convolución lineal y no a una convolución circular, necesitamos añadir suficiente zero-padding. En este caso, para la convolución lineal asumimos que la imagen está extendida con ceros fuera de sus límites, de modo que el filtro simplemente “sale” por los bordes. En una convolución circular, en cambio, la imagen se considera periódica y lo que sale por un borde reaparece por el opuesto. El tamaño mínimo para calcular la convolución lineal completa es H + M - 1 por W + M - 1.

Este padding se introduce directamente mediante el argumento s de np.fft.fft2. Por tanto, tanto la imagen como el filtro se amplían con ceros hasta fft_shape antes de calcular sus transformadas de Fourier.

A continuación, multiplicamos ambas transformadas punto a punto. Según el teorema de la convolución, este producto es la transformada de Fourier de la convolución entre la imagen y el filtro.

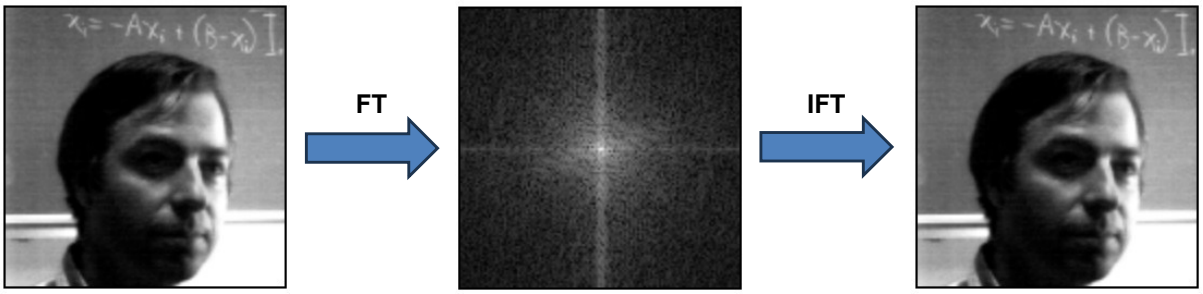
Después, aplicamos la transformada inversa mediante ifft2 y obtenemos la convolución completa. Como esta tiene un tamaño mayor que la imagen original, recortamos la región central de tamaño H x W para obtener una salida de tipo same.

Finalmente, extraemos la parte real del resultado. Teóricamente, como tanto la

imagen como el filtro son reales, el resultado también debe ser real.

Fourier Analysis in Images

An inverse Fourier transform of the Fourier image produces an exact pixel-for-pixel replica of the original brightness image.



https://web.archive.org/web/20130513181427id_/http://sharp.bu.edu/~slehar/fourier/fourier.html#filtering

56 of 90

Partimos de una imagen en el dominio espacial y calculamos su transformada de Fourier. El resultado contiene toda la información necesaria para reconstruir la imagen original, aunque ahora esté representada en términos de componentes frecuenciales.

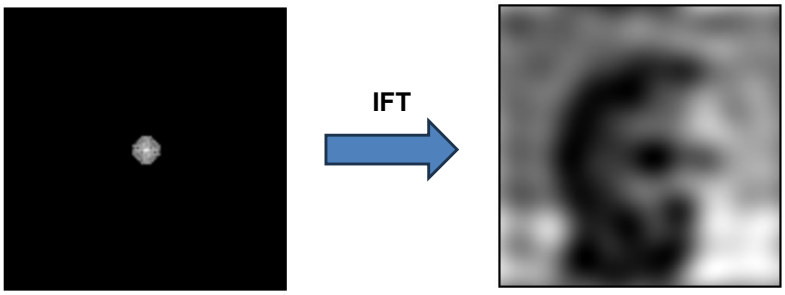
Si conservamos todos los coeficientes complejos de Fourier, es decir, tanto su magnitud como su fase, podemos aplicar la transformada inversa de Fourier y recuperar exactamente la imagen original, salvo los pequeños errores numéricos propios de la representación en coma flotante.

Por tanto, la transformada de Fourier es simplemente otra representación de la misma información.

Fourier Analysis in Images

Low frequency components are found near the central DC point → we have to define a radius around the DC point, and zero-out everything else.

The low-pass filtered transform is identical to the central portion of the Fourier transform.



https://web.archive.org/web/20130513181427id_/http://sharp.bu.edu/~slehar/fourier/fourier.html#filtering

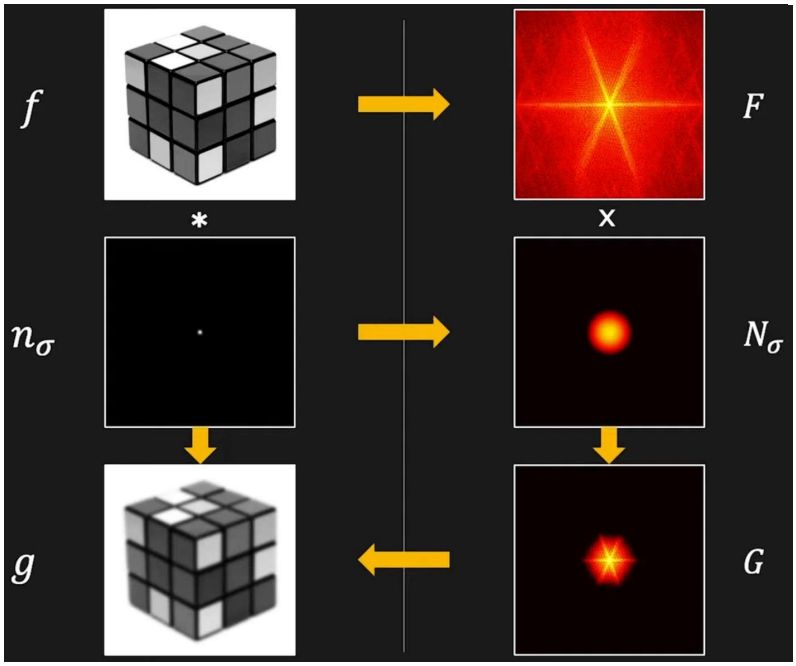
57 of 90

Como hemos visto, después de centrar el espectro, las bajas frecuencias se encuentran alrededor de la componente DC, en el centro de la transformada. Por tanto, una forma sencilla de construir un filtro paso bajo ideal consiste en definir una región alrededor del centro y conservar únicamente los coeficientes de Fourier que caen dentro de esa región, poniendo a cero todos los demás.

En la imagen de la izquierda vemos precisamente el resultado de esa operación: solo conservamos una pequeña zona central del espectro. Al aplicar después la transformada inversa de Fourier recuperamos una imagen en el dominio espacial en la que han desaparecido gran parte de los detalles finos y de los cambios bruscos de intensidad. El resultado es, por tanto, una imagen fuertemente suavizada.

Cuanto menor sea el radio de la región que conservamos alrededor del centro, más agresivo será el filtrado paso bajo, porque estaremos eliminando una mayor cantidad de frecuencias.

Fourier Analysis in Images



Slide:
Shree Nayar

58 of 90

En esta diapositiva vemos de forma gráfica cómo el filtrado gaussiano puede realizarse indistintamente en el dominio espacial o en el dominio de Fourier.

En la parte izquierda partimos de la imagen f y de un filtro gaussiano n_sigma. Si convolucionamos ambos en el dominio espacial obtenemos la imagen suavizada g.

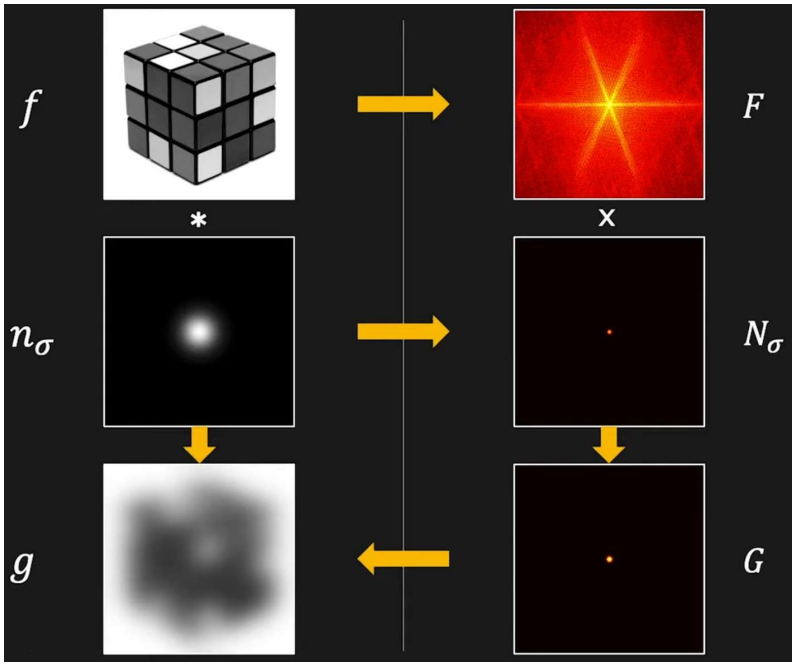
En la parte derecha tenemos exactamente la operación equivalente en el dominio de Fourier. Calculamos la transformada de Fourier de la imagen, F, y la transformada de Fourier del filtro gaussiano, N_sigma. Una gaussiana sigue siendo una gaussiana al pasar al dominio de Fourier. Además, su transformada tiene valores elevados alrededor de la frecuencia cero y decrece progresivamente conforme nos alejamos del centro, por lo que se comporta como un filtro paso bajo.

En lugar de realizar una convolución, en el dominio de Fourier multiplicamos F y N_sigma punto a punto. Como consecuencia, las bajas frecuencias de la imagen se conservan en mayor medida, mientras que las frecuencias más altas se atenúan progresivamente. El resultado de esta multiplicación es G.

Finalmente, aplicando la transformada inversa de Fourier a G recuperamos g en el dominio espacial. Es exactamente la misma imagen que obtenemos convolucionando directamente f con la gaussiana.

La diapositiva resume, por tanto, el teorema de la convolución aplicado al filtrado: convolucionar en el dominio espacial equivale a multiplicar punto a punto en el dominio de Fourier.

Fourier Analysis in Images



Slide:
Shree Nayar

59 of 90

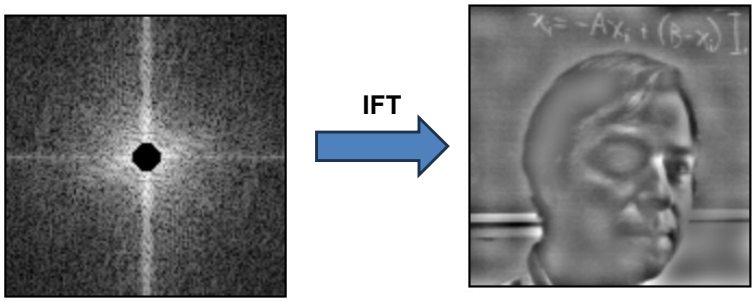
En esta diapositiva vemos el mismo esquema que en la anterior, pero utilizando una gaussiana mucho más ancha en el dominio espacial.

En la parte izquierda, la gaussiana n_sigma tiene una sigma mayor, de modo que el suavizado que aplica sobre la imagen f es mucho más intenso. Por eso, la imagen resultante g aparece fuertemente difuminada y apenas conserva detalles finos.

Esta diapositiva muestra además una relación importante: cuanto mayor es sigma en el dominio espacial, más estrecha es la gaussiana en el dominio de Fourier y, por tanto, más agresivo es el filtrado paso bajo. Esto nos ayuda a entender visualmente por qué un sigma mayor produce un suavizado más intenso. Al aumentar sigma, la gaussiana se hace más ancha y más dispersa en el dominio espacial. Sin embargo, en el dominio de Fourier ocurre lo contrario: su transformada se hace más estrecha y queda más concentrada alrededor de la frecuencia cero. Esto significa que, al multiplicarla por la transformada de Fourier de la imagen, dejamos pasar un rango más reducido de frecuencias. Conservamos fundamentalmente las frecuencias más bajas y atenuamos con mayor intensidad las frecuencias medias y altas. Como consecuencia, se elimina más detalle de la imagen y obtenemos un suavizado más fuerte. Por tanto, existe una relación inversa entre la anchura de la gaussiana en ambos dominios: cuanto más ancha es en el dominio espacial, más estrecha es en el dominio frecuencial.

Fourier Analysis in Images

Same idea for high-pass filtering. All spatial frequency components that fall within that radius are eliminated, preserving only the higher spatial frequency components



https://web.archive.org/web/20130513181427id_/http://sharp.bu.edu/~slehar/fourier/fourier.html#filtering

60 of 90

En esta diapositiva aplicamos la misma idea, pero ahora para construir un filtro paso alto en el dominio de Fourier.

Como las bajas frecuencias se encuentran alrededor de la componente DC, podemos eliminar todas las componentes que estén dentro de un determinado radio alrededor del centro y conservar las que quedan fuera.

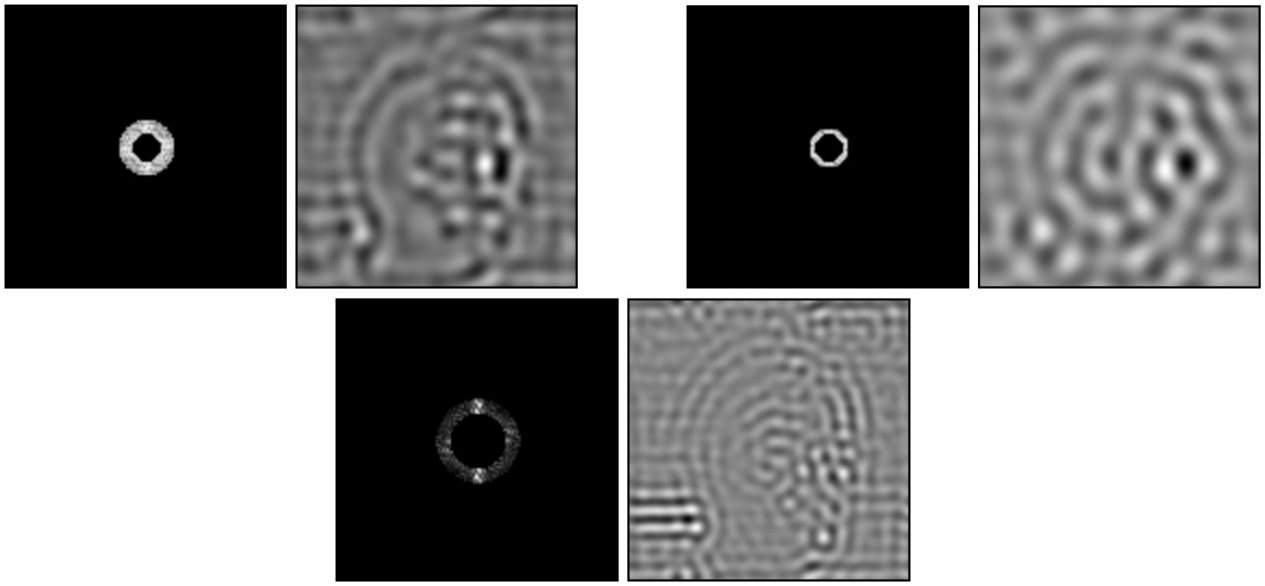
De esta forma eliminamos las variaciones suaves y la estructura global de la imagen, que están asociadas principalmente a las bajas frecuencias, y conservamos las componentes de mayor frecuencia, asociadas a cambios rápidos de intensidad, bordes y detalles finos.

Al aplicar después la transformada inversa de Fourier obtenemos la imagen que vemos a la derecha, en la que predominan precisamente los contornos y las transiciones de intensidad.

Cuanto mayor sea el radio de la región central que eliminamos, mayor será el rango de bajas y medias frecuencias que descartamos y más restrictivo será el filtrado paso alto.

Fourier Analysis in Images

Same for band-pass filters…



https://web.archive.org/web/20130513181427id_/http://sharp.bu.edu/~slehar/fourier/fourier.html#filtering

61 of 90

En esta diapositiva extendemos exactamente la misma idea a los filtros paso banda.

En lugar de conservar únicamente las frecuencias cercanas a la componente DC, como en un paso bajo, o únicamente las frecuencias alejadas del centro, como en un paso alto, ahora conservamos solamente un determinado intervalo de frecuencias.

En el dominio de Fourier esto se representa mediante un anillo: ponemos a cero las frecuencias que quedan dentro del radio interior y también las que quedan fuera del radio exterior. De este modo, solo permanecen las frecuencias intermedias comprendidas entre ambos radios.

Al aplicar la transformada inversa de Fourier obtenemos una imagen que contiene principalmente estructuras asociadas a esa banda concreta de frecuencias espaciales.

Las distintas imágenes de la diapositiva muestran precisamente cómo cambia el resultado cuando seleccionamos bandas diferentes: una banda más próxima al centro conserva estructuras de frecuencia relativamente baja, mientras que una banda situada más lejos del centro conserva estructuras más finas y cambios espaciales más rápidos.

Fourier Analysis in Images

- LoG and DoG are **band-pass filters**!

$L = \sigma^2\left(G_{xx}(x,y,\sigma) + G_{yy}(x,y,\sigma)\right)$

(Laplacian)

$DoG = G(x,y,k\sigma) - G(x,y,\sigma)$

(Difference of Gaussians)

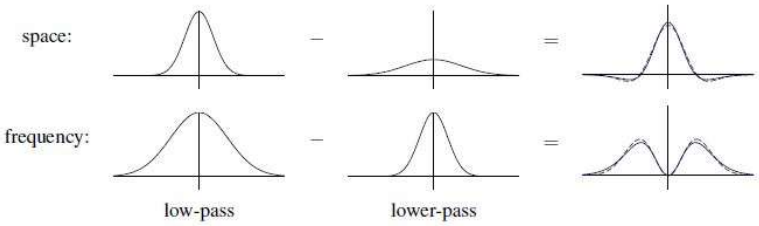


Figure 3.35 The difference of two low-pass filters results in a band-pass filter. The dashed blue lines show the close fit to a half-octave Laplacian of Gaussian.

https://maucher.pages.mi.hdm-stuttgart.de/orbook/preprocessing/05GaussianLowPassFilter.html

The Gaussian filters involved are "killing" different high-frequencies. Their subtraction gives us a range/band of frequencies whose pass is allowed!

62 of 90

En esta diapositiva vemos por qué tanto el LoG como el DoG se comportan como filtros paso banda. Recordemos que la DoG es una aproximación de la LoG.

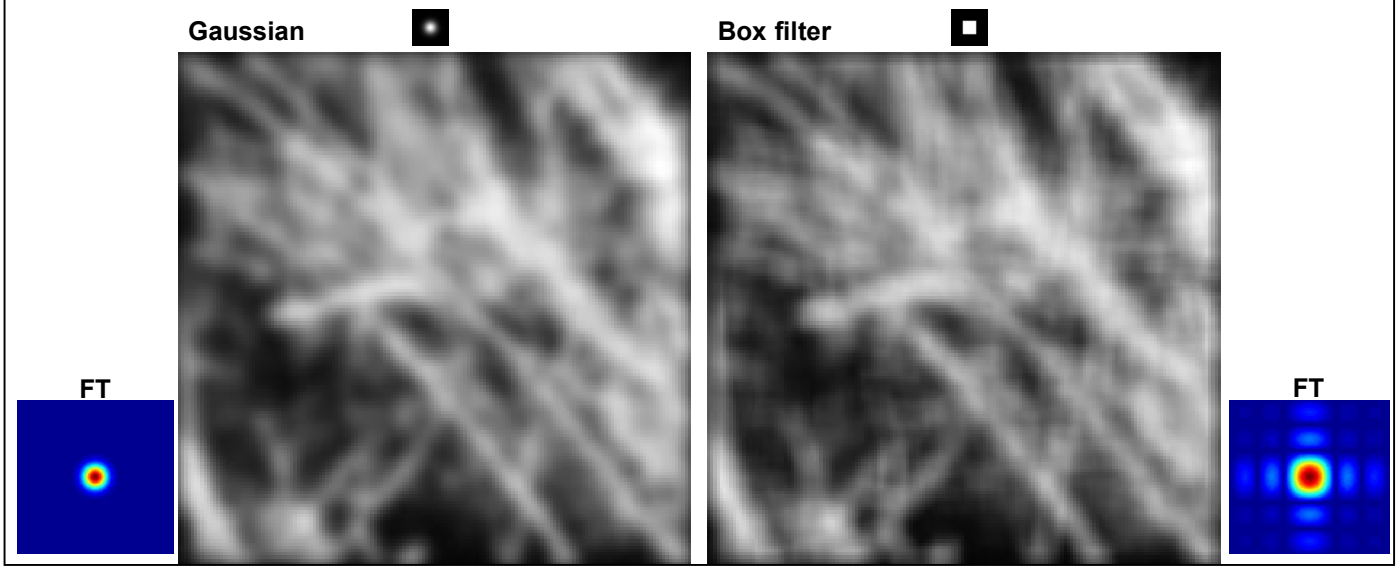
En el caso de la DoG, partimos de dos gaussianas con distinta sigma. En el dominio espacial, una es más estrecha y la otra más ancha. Al pasar al dominio de Fourier ocurre lo contrario: obtenemos dos respuestas paso bajo con distinta anchura en frecuencia. Una deja pasar un rango más amplio de frecuencias y la otra un rango más reducido.

Cuando restamos ambas respuestas frecuenciales, las bajas frecuencias comunes a las dos se cancelan, mientras que queda una determinada banda de frecuencias intermedias. Por eso la diferencia de gaussianas se comporta como un filtro paso banda: atenúa las frecuencias muy bajas, deja pasar una banda intermedia y vuelve a atenuar las frecuencias más altas.

Es importante observar que las gaussianas están normalizadas. Como la componente de frecuencia cero de la transformada de Fourier es igual a la integral de la función, dos gaussianas normalizadas a área 1 tienen exactamente la misma amplitud en DC, aunque tengan diferente sigma. Por eso, al restar sus transformadas, la componente DC se cancela exactamente. Como sus anchuras en frecuencia son diferentes, la resta deja una banda de frecuencias intermedias.

Why does the Gaussian give a nice smooth image, but the square filter give edgy artifacts?

Let's try to think a bit in frequency terms!



Y ahora vamos a intentar dar respuesta a una cuestión que planteamos cuando hablamos de suavizado de imágenes, y comparamos el efecto de suavizado de una gaussiana y lo comparamos con un box filter. En esta diapositiva y la siguiente intentamos entender lo que ocurre a nivel de frecuencias.

En el dominio espacial, la gaussiana tiene una transición suave desde el centro hacia los valores más pequeños. Su transformada de Fourier también es una gaussiana, de modo que la respuesta en frecuencia decrece suavemente conforme aumenta la frecuencia. Por tanto, las altas frecuencias se atenúan de forma progresiva, sin introducir transiciones bruscas en la respuesta frecuencial.

El filtro de caja, en cambio, tiene una discontinuidad muy fuerte en el dominio espacial: dentro de la ventana todos los coeficientes tienen el mismo valor y fuera pasan bruscamente a cero. Esa forma rectangular se transforma en una respuesta tipo sinc en frecuencia. En dos dimensiones aparece el patrón de lóbulos que vemos en la diapositiva.

Esto significa que el box filter no atenúa las frecuencias de manera monótona y suave. Su respuesta presenta ceros y lóbulos secundarios, de modo que algunas frecuencias relativamente altas siguen pasando parcialmente mientras que otras quedan mucho más atenuadas.

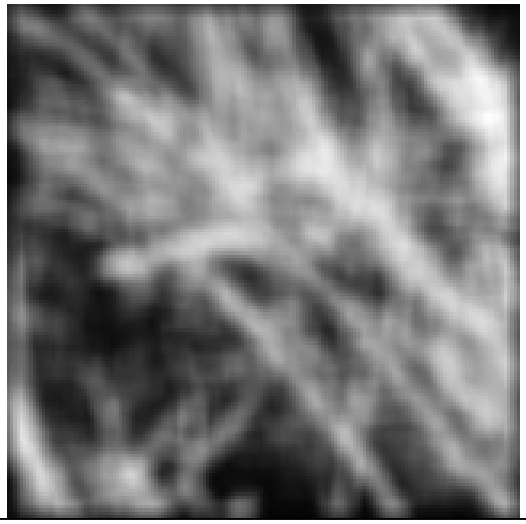
Es precisamente esa respuesta frecuencial oscilatoria la que puede producir los artefactos visibles en la imagen filtrada. La gaussiana, al tener una respuesta frecuencial también suave y sin lóbulos secundarios, produce un suavizado mucho

más natural.

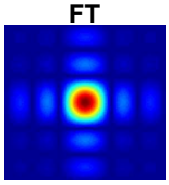
Why does the Gaussian give a nice smooth image, but the square filter give edgy artifacts?

Let's try to think a bit in frequency terms!

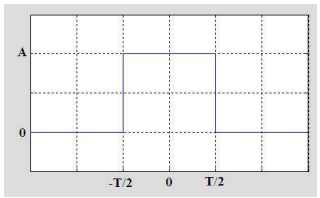
Box filter



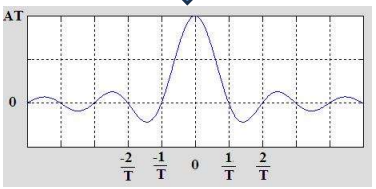
FT



This makes a lot of sense when you consider that the FT of a box function (square pulse) is the sinc function!



FT



64 of 90

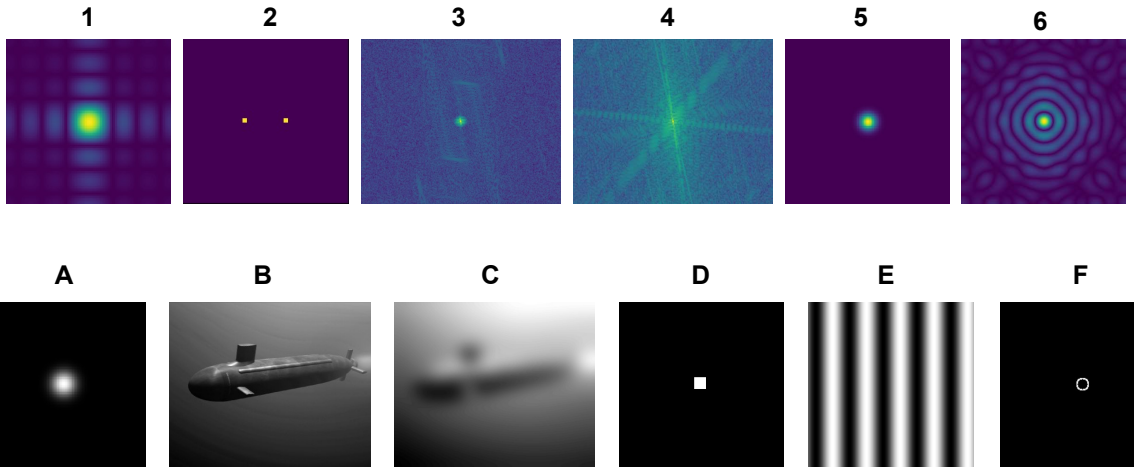
En una dimensión, un filtro de caja no es más que una función rectangular: mantiene un valor constante dentro de un determinado intervalo y vale cero fuera. La transformada de Fourier de esa función rectangular es una función sinc.

La sinc tiene un lóbulo principal alrededor de la frecuencia cero, pero también presenta una sucesión de ceros y lóbulos secundarios. Por eso, aunque el box filter se comporte globalmente como un filtro paso bajo, su atenuación de las frecuencias no es suave ni monótona.

En dos dimensiones ocurre lo mismo en cada dirección. Como el filtro de caja es separable, su transformada de Fourier es el producto de dos funciones sinc, una en cada eje. De ahí aparece el patrón de lóbulos que observamos en la representación 2D de la transformada.

Esto explica por qué el box filter puede producir artefactos: algunas frecuencias relativamente altas no se atenúan de manera uniforme, sino que reaparecen en los lóbulos secundarios de la sinc. En cambio, la gaussiana tiene una respuesta frecuencial suave, sin esos lóbulos secundarios, y por eso produce un suavizado más limpio.

Match the spatial domain image to the Fourier magnitude image



But… what about the phase???

"We generally do not display PHASE images because most people who see them shortly thereafter succomb to hallucinogenics or end up in a Tibetan monastery."

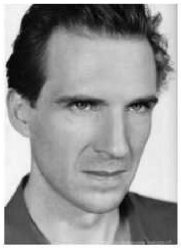
"Introduction to Fourier Transforms for Image Processing"
(by John M. Brayer, University of New Mexico):
https://www.cs.unm.edu/~brayer/vision/fourier.html

66 of 90

Hasta ahora nos hemos centrado sobre todo en la magnitud de la transformada, porque nos permite ver qué frecuencias están presentes y con qué peso. Sin embargo, cada coeficiente de Fourier es complejo y, por tanto, además de magnitud tiene una fase.

La fase contiene información fundamental sobre la organización espacial de la imagen, es decir, sobre cómo se alinean y se combinan espacialmente las distintas componentes sinusoidales. Aunque visualmente una imagen de fase suele ser mucho menos intuitiva que una imagen de magnitud, no significa en absoluto que sea menos importante.

But… what about the phase???

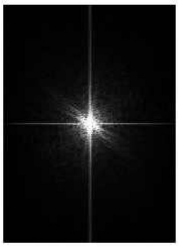


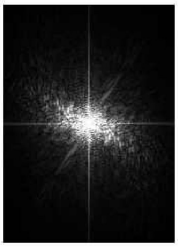


The **magnitude** tells "how much" of a certain frequency component is present.

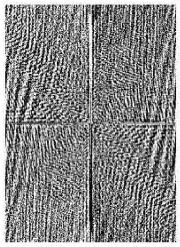
The **phase** tells "where" the frequency component is in the signal/image.

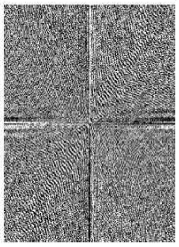
Magnitude





Phase





Slide: Deepa Kundur

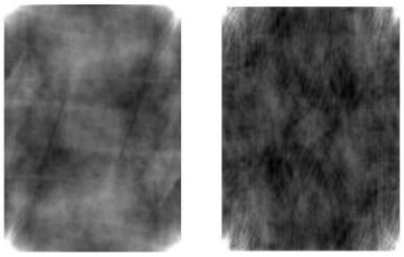
En esta diapositiva, y las siguientes, queremos entender mejor qué papel desempeña la fase de la transformada de Fourier y hasta qué punto contiene información relevante sobre la imagen.

Aquí comparamos dos imágenes diferentes junto con sus representaciones de magnitud y fase. La magnitud nos indica cuánto pesa cada componente frecuencial, mientras que la fase determina cómo se combinan espacialmente esos componentes. Por eso, aunque una imagen de fase sea visualmente difícil de interpretar directamente, contiene una parte esencial de la estructura espacial de la imagen.

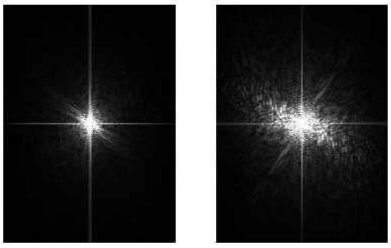
But… what about the phase???

Reconstruction using magnitude only!!!!

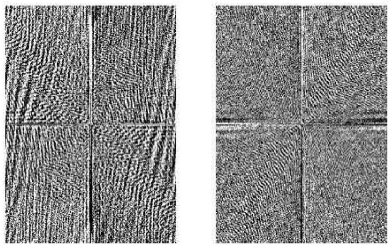
Magnitude is the same; phase=0



Magnitude



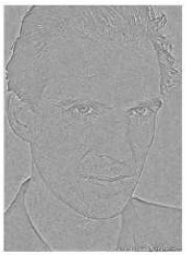
Phase



Slide: Deepa Kundur

En esta diapositiva reconstruimos las imágenes utilizando únicamente la magnitud, fijando toda la fase a cero. El resultado pierde prácticamente toda la estructura reconocible de las imágenes originales. Seguimos conservando qué frecuencias estaban presentes y con qué intensidad, pero hemos perdido la información necesaria para organizarlas espacialmente de la forma correcta.

But… what about the phase???

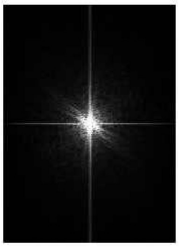


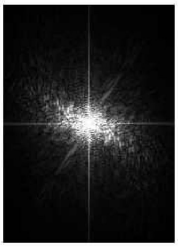


Reconstruction using phase only!!!!

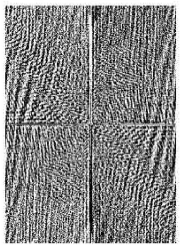
Magnitude normalized to 1; phase is the same

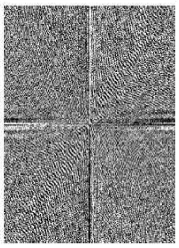
Magnitude





Phase





Slide: Deepa Kundur

Ahora hacemos el experimento complementario: conservamos la fase original y fijamos todas las magnitudes al mismo valor. Ahora sí podemos reconocer con bastante claridad la estructura de las imágenes originales, especialmente sus contornos y la organización espacial de sus elementos. Esto muestra de manera muy visual que la fase desempeña un papel fundamental en la localización y organización de las estructuras de la imagen.

But… what about the phase???

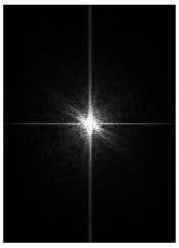
Ralph's phase + Meg's magnitude

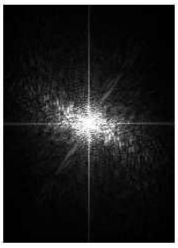




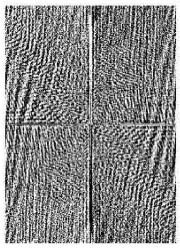
Meg's phase + Ralph's magnitude

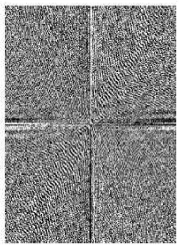
Magnitude





Phase





Slide: Deepa Kundur

Ahora intercambiamos magnitud y fase entre las dos imágenes. Cuando combinamos la fase de Ralph con la magnitud de Meg, el resultado se parece fundamentalmente a Ralph. Cuando hacemos lo contrario, utilizando la fase de Meg y la magnitud de Ralph, el resultado se parece principalmente a Meg. Es decir, la identidad estructural de la imagen viene determinada en gran medida por la fase.

But… what about the phase???

Ralph's phase + Meg's magnitude





Meg's phase + Ralph's magnitude

Which is more important for a given signal? Magnitude or phase?

Does one component (magnitude or phase) contain more information than another?

🤔

When filtering, if we had to preserve one component (magnitude or phase) more, which one would it be?

Slide: Deepa Kundur

Further Readings

(for those who want to dive deeper into Fourier analysis)

- "An Intuitive Explanation of Fourier Theory" (by Steven Lehar, Boston University): https://web.archive.org/web/20130513181427id_/http://sharp.bu.edu/~slehar/fourier/fourier.html
- "Introduction to Fourier Transforms for Image Processing" (by John M. Brayer, University of New Mexico): https://www.cs.unm.edu/~brayer/vision/fourier.html
- Heckbert, Paul. "Fourier transforms and the fast Fourier transform (FFT) algorithm." Computer Graphics 2.1995 (1995). https://es.scribd.com/doc/124484613/Fourier-Transforms-FFT-Algorithm-Paul-Heckbert-1998-by-Tantanoid
- "Fourier Transform" (by Iain Collings, Macquarie University): https://www.iaincollings.com/signals-and-systems
- "Digital Signal Processing" (by Deepa Kundur, University of Toronto): https://www.comm.utoronto.ca/dkundur/course/real-time-digital-signal-processing/ and https://www.comm.utoronto.ca/dkundur/course_info/real-time-DSP/notes/7_Kundur_FFT.pdf
- "First Principles of Computer Vision" (by Shree Nayar, Columbia University): Fourier Transform (https://www.youtube.com/watch?v=tEzgtbnbXgQ)

72 of 90

En esta diapositiva dejamos algunas referencias adicionales para quienes quieran profundizar en el análisis de Fourier y su aplicación al procesamiento de imágenes. La idea es que sirvan como recursos complementarios para quien quiera entender con más detalle tanto la transformada de Fourier como su interpretación en imágenes y señales.

Upsampling and Interpolation

Este bloque de diapositivas (es decir, este PDF) lo comenzamos con una idea intuitiva muy sencilla. Teníamos una imagen de Vincent van Gogh que era demasiado grande como para encajarla en la pantalla y queríamos, por tanto, reducir su tamaño.

A partir de ahí comenzó todo nuestro recorrido dentro de este bloque: cómo podemos reducir adecuadamente el tamaño de una imagen, qué problemas pueden aparecer al hacerlo y cómo evitarlos.

Ahora, en esta última sección, vamos a plantearnos precisamente el problema opuesto: cómo podemos aumentar el tamaño de una imagen.

Para ello necesitaremos introducir un concepto fundamental: la interpolación.

Upsampling

- This image is too small for this screen: 
- How can we make it 10 times bigger?
- Simplest approach:
  repeat each row
  and column 10 times
  + “Nearest neighbor
  interpolation”



74 of 90

En esta diapositiva introducimos la forma más sencilla de aumentar el tamaño de una imagen.

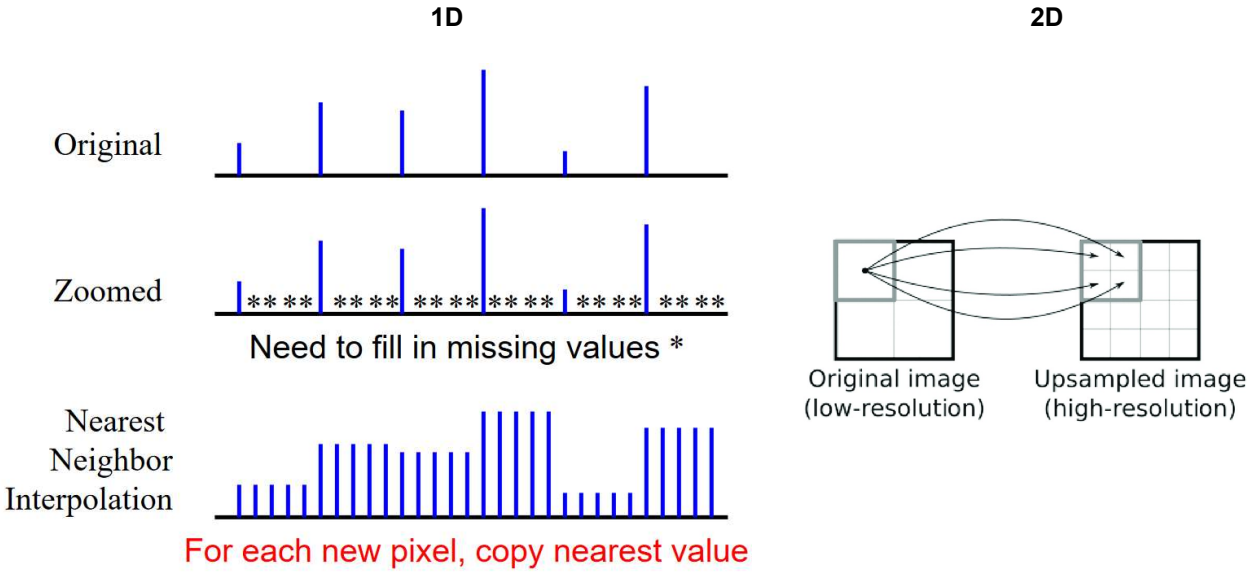
Partimos de una imagen demasiado pequeña para visualizarla adecuadamente en la pantalla y queremos hacerla, por ejemplo, diez veces más grande.

La estrategia más simple consiste en repetir cada fila y cada columna de la imagen original varias veces. Si queremos multiplicar por diez el tamaño en cada dimensión, cada píxel original pasa a ocupar un bloque de 10x10 píxeles con exactamente el mismo valor.

Este procedimiento corresponde a la interpolación por vecino más cercano, nearest-neighbor interpolation. Para cada nueva posición de la imagen ampliada, simplemente asignamos el valor del píxel original más cercano.

Es un método muy sencillo y rápido, pero el resultado tiene un aspecto claramente pixelado, porque no estamos creando transiciones nuevas entre los valores originales: simplemente estamos replicando los píxeles existentes.

Nearest Neighbor Interpolation



Source: R. Rao

En esta diapositiva vemos con un poco más de detalle qué significa la interpolación por vecino más cercano.

En una dimensión, partimos de una serie de muestras originales. Cuando aumentamos la resolución, aparecen nuevas posiciones para las que no tenemos ningún valor medido directamente. Esos son los valores que necesitamos estimar.

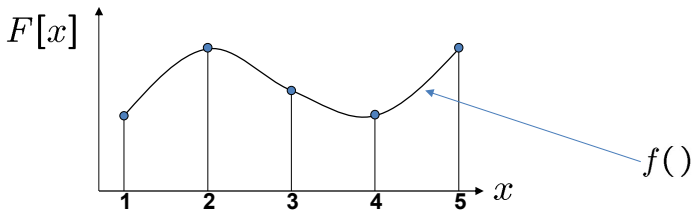
La interpolación por vecino más cercano resuelve el problema de la forma más sencilla posible: para cada nueva posición, buscamos cuál es la muestra original más cercana y copiamos exactamente su valor.

Por eso, en la representación inferior aparecen tramos constantes. Varias posiciones nuevas reciben exactamente el mismo valor hasta que pasamos a estar más cerca de la siguiente muestra original.

En dos dimensiones ocurre exactamente lo mismo. Para cada nuevo píxel de la imagen ampliada buscamos el píxel de la imagen original que queda más cerca y copiamos su intensidad.

El resultado es muy rápido de calcular, pero produce regiones constantes y transiciones abruptas entre ellas. Por eso, cuando el factor de ampliación es grande, aparece el aspecto característico de bloques o píxeles grandes.

Image interpolation



Recall that a digital image is formed as follows:

$$F[x, y] = \text{quantize}\{f(xd, yd)\}$$

d = 1 in this example

d is the sampling interval, i.e., the spacing between consecutive samples. Smaller d → higher the sampling frequency.

- It is a discrete point-sampling of a continuous function
- **If we could somehow reconstruct the original function, any new image could be generated, at any resolution and scale**

Adapted from: S. Seitz

Ahora damos un paso más y conectamos la interpolación con la idea de que una imagen digital puede verse como un conjunto de muestras de una función continua subyacente.

En una dimensión, por ejemplo, tenemos una función continua f(x), pero su representación discreta solo conserva valores en determinadas posiciones discretas. Esos valores son las muestras que aparecen como puntos en la gráfica.

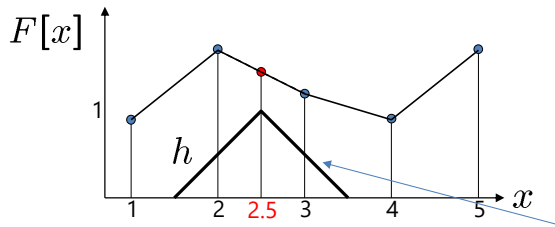
En una imagen ocurre lo mismo en dos dimensiones. Podemos pensar que existe una función continua f(x,y) y que la imagen digital F[x,y] se obtiene muestreando esa función en una rejilla discreta y cuantizando después sus valores.

La idea clave de la interpolación es la siguiente: si pudiéramos reconstruir, aunque fuese aproximadamente, esa función continua subyacente a partir de las muestras disponibles, entonces podríamos volver a muestrearla en cualquier nueva rejilla.

Por tanto, aumentar la resolución de una imagen consiste esencialmente en estimar qué valores tendría esa función entre las muestras originales y evaluar después esa estimación en nuevas posiciones.

Eso es precisamente lo que hacen los distintos métodos de interpolación. La diferencia entre vecino más cercano, interpolación bilineal, bicúbica, etc., está en cómo estimamos esa función continua entre las muestras conocidas.

Image interpolation



F=[2 0 1.5]
h = [0.5 1 0.5]
F*h = 1 + 0.75 = 1.75

- But… we don't know f 😔
  - Guess an approximation: $\tilde{f}$
  - Can be done in a **principled way: filtering**
  - Convert F to a continuous function:

$$f_F(x) = F(\tfrac{x}{d}) \text{ when } \tfrac{x}{d} \text{ is an integer, 0 otherwise}$$

d = 1 in this example

  - **Reconstruct by convolution with a *reconstruction filter*, h**

$$\tilde{f} = h * f_F$$

Adapted from: Snavely

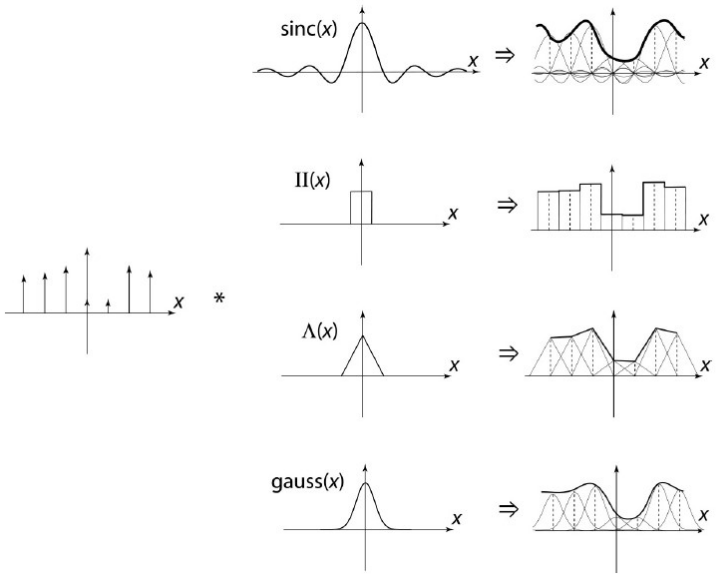
En esta diapositiva damos una interpretación más formal de la interpolación como un problema de reconstrucción mediante filtrado.

Partimos únicamente de las muestras originales F, pero desconocemos la función continua f de la que proceden. La idea consiste en construir primero una representación en la que mantenemos los valores conocidos en sus posiciones originales y dejamos a cero las posiciones intermedias. A partir de ahí utilizamos un filtro de reconstrucción h para estimar los valores que faltan.

La reconstrucción se obtiene convolucionando esa representación de las muestras con el filtro h. Cada muestra original contribuye así a las posiciones de su entorno siguiendo la forma del filtro, y el valor interpolado en una nueva posición resulta de sumar todas esas contribuciones.

En el ejemplo queremos estimar el valor en x = 2.5. El filtro triangular h asigna un peso de 0.5 a las dos muestras vecinas, cuyos valores son 2 y 1.5. Por tanto, el valor interpolado es 0.5 por 2 más 0.5 por 1.5, es decir, 1.75. Esto corresponde precisamente a una interpolación lineal entre ambas muestras.

Image interpolation



Source: B. Curless

Aquí vemos distintas formas de reconstruir una señal continua a partir de sus muestras, dependiendo del filtro de reconstrucción que utilicemos.

La idea general es siempre la misma: representamos las muestras originales como valores situados en posiciones discretas y después convolucionamos esa señal muestreada con un determinado kernel de reconstrucción. Cada muestra genera una copia desplazada del kernel, escalada por el valor de esa muestra, y la suma de todas esas contribuciones produce la señal reconstruida.

Con la función sinc podemos obtener la reconstrucción ideal, ya que la sinc es el kernel de reconstrucción correspondiente a un filtro paso bajo ideal, cuya respuesta en frecuencia es rectangular.

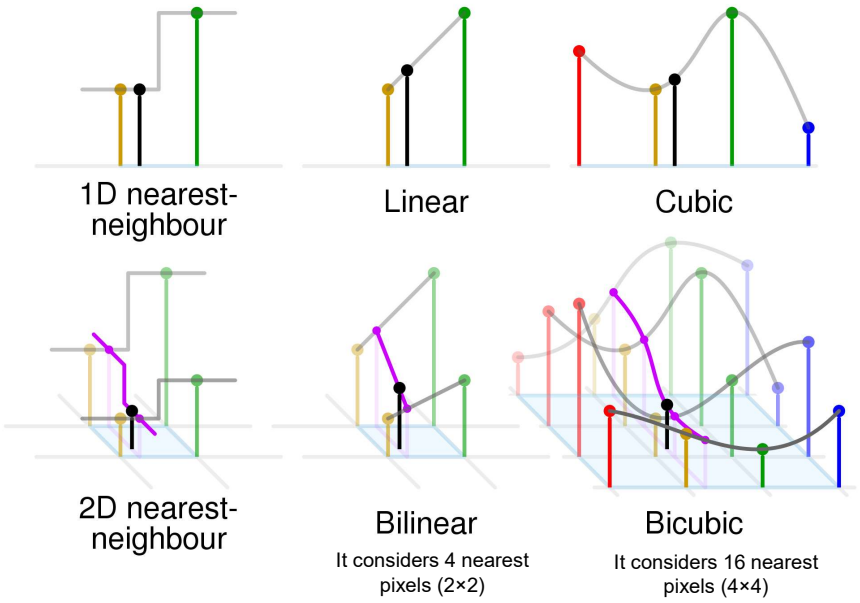
Con un kernel rectangular obtenemos la interpolación por vecino más cercano. Cada muestra mantiene su valor constante en una determinada región de su entorno, produciendo una reconstrucción a trozos constante.

Con un kernel triangular obtenemos interpolación lineal. Entre dos muestras consecutivas, el valor reconstruido cambia linealmente de una a otra.

Finalmente, también podemos utilizar una gaussiana como filtro de reconstrucción. En ese caso cada muestra contribuye suavemente a su entorno y obtenemos una reconstrucción más suave.

La diapositiva muestra, por tanto, que diferentes métodos de reconstrucción pueden entenderse de una forma unificada: cambian fundamentalmente en el filtro de reconstrucción que utilizamos.

Image interpolation



Deep learning approaches are also successfully used in image scaling. See Wang, Xintao, et al. "ESRGAN: Enhanced super-resolution generative adversarial networks." *Proceedings of the European conference on computer vision (ECCV) workshops*. 2018.

Black and red/yellow/green/blue dots correspond to the **interpolated point** and neighboring samples, respectively. Their heights above the ground correspond to their values.

https://en.wikipedia.org/wiki/Nearest-neighbor_interpolation#/media/File:Comparison_of_1D_and_2D_interpolation.svg

79 of 90

En esta diapositiva comparamos varias estrategias de interpolación, primero en una dimensión y después en dos dimensiones.

En 1D, el vecino más cercano asigna a cada nueva posición el valor de la muestra original más próxima, por eso obtenemos una función a trozos constante. La interpolación lineal conecta cada par de muestras consecutivas mediante una recta. La interpolación cúbica utiliza polinomios de tercer grado y permite obtener una reconstrucción más suave.

En 2D aparecen los análogos correspondientes. El vecino más cercano selecciona simplemente el píxel original más próximo. La interpolación bilineal utiliza los cuatro píxeles vecinos de una región 2 por 2 y combina sus valores mediante interpolación lineal en las dos direcciones. La interpolación bicúbica utiliza un entorno mayor, típicamente 4 por 4, es decir, 16 píxeles, y genera transiciones más suaves.

La figura ilustra precisamente cómo cambia el valor interpolado según el método utilizado. Los puntos de colores representan las muestras vecinas y el punto negro representa la posición cuyo valor queremos estimar.

Finalmente, la diapositiva recuerda que existen también métodos de aprendizaje profundo para aumentar la resolución de imágenes. En ese caso ya no hablamos simplemente de interpolar a partir de vecinos mediante una regla fija, sino de utilizar un modelo aprendido a partir de datos para estimar una imagen de mayor resolución.

Image interpolation

What would you say is the main effect yielded by the following 2D filter?

$$\begin{matrix} 1/4 & 1/2 & 1/4 \\ 1/2 & 1 & 1/2 \\ 1/4 & 1/2 & 1/4 \end{matrix}$$

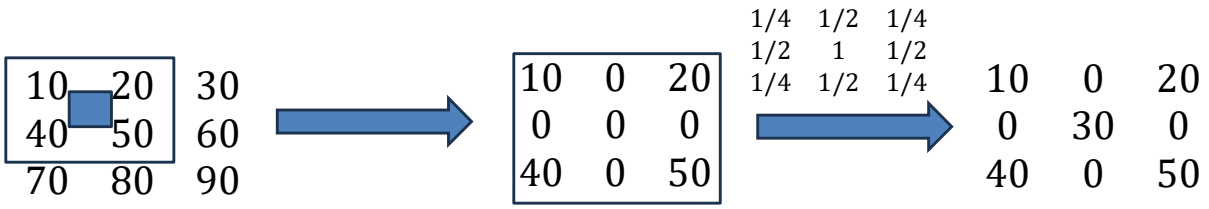


(a) Gaussian Smoothing
(b) Edge Detection
(c) Bilinear Interpolation
(d) None of the Above

Image interpolation

What would you say is the main effect yielded by the following 2D filter?

$$\begin{matrix} 1/4 & 1/2 & 1/4 \\ 1/2 & 1 & 1/2 \\ 1/4 & 1/2 & 1/4 \end{matrix}$$



81 of 90

En esta diapositiva vemos otra forma de entender la interpolación lineal, ahora directamente como una operación de filtrado en dos dimensiones.

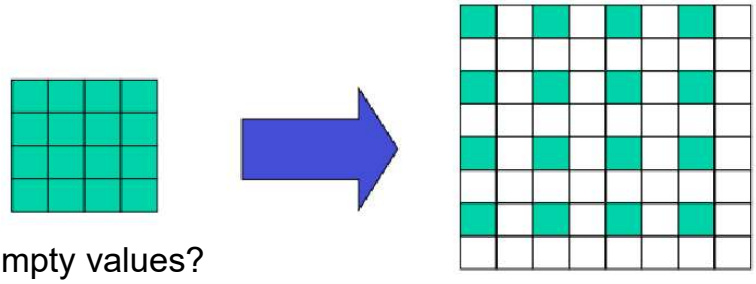
Partimos de una imagen de baja resolución. El primer paso consiste en insertar ceros entre los píxeles originales, tanto en horizontal como en vertical. De este modo aumentamos el tamaño de la rejilla, pero todavía no hemos estimado los valores de los nuevos píxeles.

A continuación, aplicamos el filtro 2D que aparece en la parte superior. Este filtro asigna peso 1 al píxel original central, peso 1/2 a sus vecinos horizontales y verticales, y peso 1/4 a los vecinos diagonales.

El efecto es que los ceros introducidos entre las muestras originales se rellenan mediante combinaciones lineales de los valores vecinos. Por tanto, este filtro implementa precisamente una interpolación bilineal sobre la rejilla ampliada: primero insertamos huecos y después los rellenamos mediante un filtro de reconstrucción separable.

Además, el kernel puede escribirse como el producto exterior del vector [1/2, 1, 1/2] consigo mismo, por lo que es separable.

Upsampling (technical note)



How to fill in the empty values?

- The empty pixels are initially set to 0
- Convolve upsampled image with Gaussian filter
- In this example, we must also multiply by 4. Why??

$$\frac{1}{256}\begin{bmatrix} 1 & 4 & 6 & 4 & 1 \\ 4 & 16 & 24 & 16 & 4 \\ 6 & 24 & 36 & 24 & 6 \\ 4 & 16 & 24 & 16 & 4 \\ 1 & 4 & 6 & 4 & 1 \end{bmatrix}$$

Gaussian filter is a smoothing filter. It sums to 1.
¾ of the new image was initially 0. We need to "recover" intensity!
Example: we have a pixel with a value of 100 → we convert it into [100, 0, 0, 0]. The average of that area is now 25. An external scaling factor (x4) is needed to restore the original brightness.

This is the **procedure followed by OpenCV (`cv2.pyrUp()`)**: https://docs.opencv.org/3.4/d4/d1f/tutorial_pyramids.html

82 of 90

En esta nota técnica mostramos una forma práctica de implementar el upsampling mediante inserción de ceros seguida de filtrado.

Partimos de una imagen y queremos duplicar su tamaño en cada dimensión. Para ello construimos primero una rejilla de mayor resolución en la que colocamos los píxeles originales en posiciones alternas y ponemos a cero las nuevas posiciones intermedias. Como duplicamos tanto horizontal como verticalmente, solo una de cada cuatro posiciones contiene inicialmente un valor original y las otras tres contienen cero.

A continuación, convolucionamos esa imagen ampliada con un filtro gaussiano. El objetivo del filtro es distribuir la contribución de cada muestra original hacia las nuevas posiciones y producir una reconstrucción suave.

Sin embargo, al haber introducido tantos ceros, la intensidad media cae. En este caso concreto, al duplicar el tamaño en las dos dimensiones, la densidad de muestras originales se reduce por un factor de cuatro. Por eso necesitamos compensar multiplicando el resultado por 4.

La intuición puede verse con un ejemplo sencillo. Si un píxel de valor 100 se transforma inicialmente en un bloque equivalente a [100, 0; 0, 0], la media pasa de 100 a 25. El factor 4 permite restaurar la escala de intensidad original después del filtrado.

Este es precisamente el principio que utiliza cv2.pyrUp: inserción implícita de ceros, filtrado con el kernel gaussiano apropiado y compensación del factor de escala.

Summary of OpenCV functions (technical note)

- **pyrUp**: Upsamples an image and then blurs it.
  - https://docs.opencv.org/3.4/d4/d1f/tutorial_pyramids.html
  - https://docs.opencv.org/3.4/d4/d86/group__imgproc__filter.html#gada75b59bdaaca411ed6fee10085eb784
  - It can be used to build the Laplacian Pyramid
- **pyrDown**: Blurs an image and downsamples it.
  - https://docs.opencv.org/3.4/d4/d86/group__imgproc__filter.html#gaf9bba239dfca11654cb7f50f889fc2ff
  - It can be used to build a Gaussian Pyramid
- **resize**.
  - https://docs.opencv.org/3.4/da/d54/group__imgproc__transform.html#ga47a974309e9102f5f08231edc7e7529d
  - Upsampling: It does not explicitly insert zeros and convolve the resulting image with a fixed reconstruction kernel. Instead, it directly computes pixel values on the target grid using the selected interpolation method (nearest, linear, cubic, Lanczos, etc.).
  - Downsampling: The behavior depends on the interpolation method. INTER_AREA is recommended for decimation because it effectively averages source pixel areas, whereas methods such as nearest-neighbor do not perform anti-aliasing filtering.

83 of 90

pyrUp aumenta el tamaño de una imagen siguiendo el esquema que acabamos de ver: expande la rejilla, inserta implícitamente nuevas posiciones y aplica el filtrado gaussiano correspondiente. Es una operación especialmente útil en la construcción y reconstrucción de pirámides, por ejemplo, en la pirámide laplaciana.

pyrDown realiza la operación complementaria: primero suaviza la imagen mediante un filtro gaussiano y después reduce su resolución. Ese filtrado previo es precisamente lo que ayuda a evitar aliasing antes del submuestreo. Por eso pyrDown es una operación natural para construir una pirámide gaussiana.

resize, en cambio, es una función más general para cambiar el tamaño de una imagen. En upsampling utiliza directamente el método de interpolación que seleccionemos, por ejemplo, nearest neighbor, bilinear, bicubic o Lanczos. No sigue necesariamente el procedimiento explícito de insertar ceros y después convolucionar.

En downsampling, resize puede incorporar el efecto de un filtrado paso bajo a través del propio método de interpolación. Por ejemplo, INTER_AREA está especialmente pensado para reducción de tamaño y realiza un promediado ponderado por área.

Image interpolation

Original image:  **x 10**



Nearest-neighbor interpolation



Bilinear interpolation



Bicubic interpolation (common choice)

84 of 90

En esta diapositiva comparamos visualmente el efecto de distintos métodos de interpolación cuando ampliamos una imagen por un factor 10.

Con vecino más cercano, cada píxel original se replica sobre una región de la imagen ampliada. Por eso aparecen bloques claramente visibles y el resultado tiene un aspecto muy pixelado.

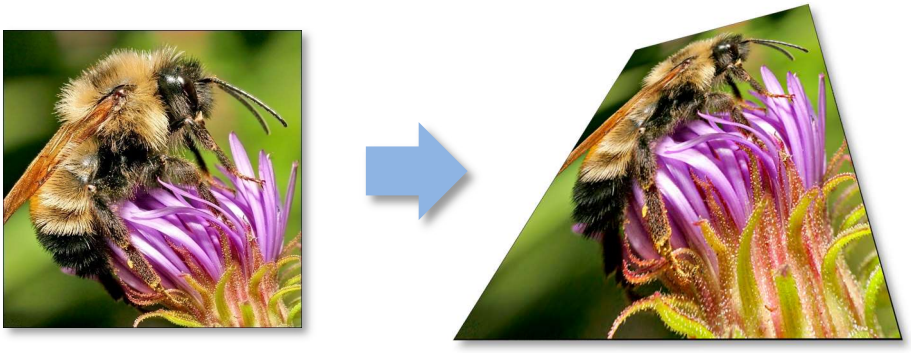
Con interpolación bilineal, cada nuevo píxel se obtiene a partir de una combinación de los cuatro vecinos más cercanos. Esto elimina gran parte de ese aspecto cuadriculado y produce transiciones más suaves, aunque también introduce cierto desenfoque.

Con interpolación bicúbica utilizamos un entorno mayor, típicamente de 4 por 4 píxeles, y una interpolación cúbica. El resultado suele ser todavía más suave y visualmente más natural que con bilineal.

La comparación permite ver una idea importante: al ampliar una imagen no estamos recuperando detalle que no existía en la imagen original. Los distintos métodos simplemente estiman de manera diferente los valores de los nuevos píxeles. Por eso una imagen ampliada puede verse más suave o menos pixelada, pero no contiene necesariamente más información real.

Image interpolation

Also used for *resampling*



Geometric transformations generally map output pixels to non-integer locations in the input image, requiring interpolation to estimate their values.

85 of 90

La interpolación no se utiliza únicamente para cambiar el tamaño de una imagen, sino también para realizar resampling después de aplicar una transformación geométrica.

Cuando aplicamos, por ejemplo, una rotación o una transformación proyectiva, los píxeles de la imagen de salida siguen estando definidos sobre una rejilla discreta de coordenadas enteras. Sin embargo, si para cada uno de esos píxeles buscamos mediante la transformación inversa de qué posición de la imagen original procede, esa posición suele tener coordenadas no enteras.

Por ejemplo, imaginemos que queremos calcular el píxel de salida situado en la posición (20, 30). Al aplicar la transformación inversa, podemos obtener que ese píxel corresponde a la posición (12.4, 17.7) de la imagen original.

El problema es que en la imagen original no existe ningún píxel exactamente en la posición (12.4, 17.7). Solo conocemos los valores de intensidad en las posiciones enteras cercanas, por ejemplo (12,17), (12,18), (13,17) y (13,18).

Ahí es donde entra la interpolación. A partir de esos valores conocidos estimamos qué intensidad tendría la imagen en la posición continua (12.4, 17.7) y asignamos ese valor al píxel correspondiente de la imagen de salida.

Dependiendo del método de interpolación que utilicemos, obtendremos una estimación diferente y, por tanto, una reconstrucción ligeramente distinta de la imagen transformada.

Así que la interpolación es una herramienta fundamental no solo para upsampling,

sino también, en general, para cualquier operación de resampling asociada a transformaciones geométricas.

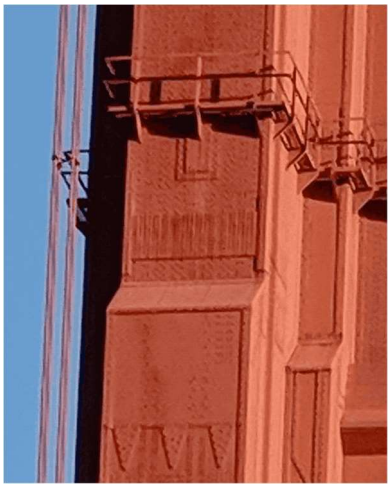
Super-resolution with multiple images

- Can do better upsampling if you have multiple images of the scene taken with small shifts
- Some cellphone cameras (like the Google Pixel line) capture a burst of photos
- Can we use that burst for upsampling?

En esta diapositiva, y en las dos siguientes, introducimos una idea diferente a la interpolación clásica: la superresolución utilizando múltiples imágenes.

Hasta ahora hemos visto cómo aumentar el tamaño de una única imagen estimando nuevos valores a partir de los píxeles disponibles. Pero si disponemos de varias imágenes de la misma escena tomadas con pequeños desplazamientos entre ellas, podemos aprovechar que cada una contiene muestras ligeramente diferentes de la escena. Esos pequeños desplazamientos hacen que distintos píxeles del sensor observen posiciones ligeramente distintas, y esa información complementaria puede utilizarse para reconstruir una imagen con mayor resolución.

Google Pixel 3 Super Res Zoom



Example photo with and without super res zoom (smart burst align and merge)

This technique merges many frames directly onto a higher resolution picture *(multi-frame super-resolution)*

https://ai.googleblog.com/2018/10/see-better-and-further-with-super-res.html

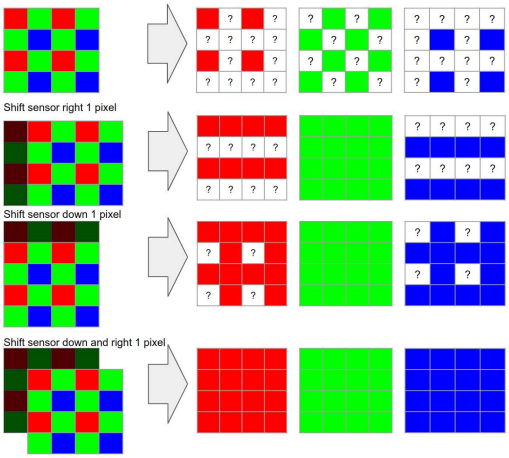
87 of 90

Esta es precisamente la idea de la superresolución multi-frame. Algunas cámaras de teléfonos móviles capturan ráfagas de imágenes en lugar de utilizar una única fotografía. Si somos capaces de alinear correctamente esas imágenes, podemos combinar la información de todas ellas para obtener una reconstrucción de mayor resolución y, al mismo tiempo, reducir ruido.

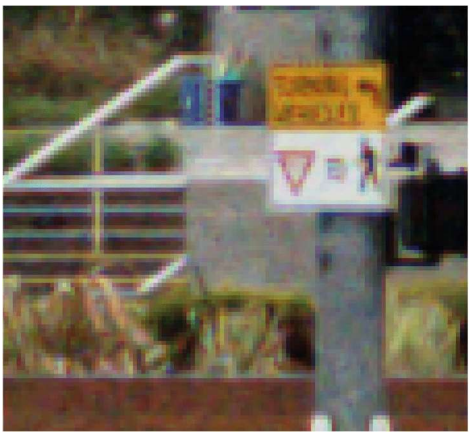
El ejemplo del Google Pixel 3 ilustra esta estrategia. En lugar de depender únicamente de un zoom digital basado en interpolación, el sistema captura varias imágenes y las combina directamente para producir una imagen de mayor resolución. Los pequeños movimientos naturales de la mano, que en principio podrían parecer un inconveniente, pueden convertirse aquí en una ventaja porque proporcionan precisamente esos pequeños desplazamientos subpíxel entre las distintas capturas.

Google Pixel 3 Super Res Zoom

Example of how an idealized multi-frame super-resolution algorithm might work:



As compared to the standard demosaicing pipeline that needs to interpolate the missing colors (top), ideally, one could fill some holes from multiple images, each shifted by one pixel horizontally or vertically.



We can take advantage of the effect of hand tremor (in this example, as seen in a cropped burst of photos, after global alignment)

https://ai.googleblog.com/2018/10/see-better-and-further-with-super-res.html

88 of 90

Aquí se muestra de forma idealizada cómo funciona esta idea. Si desplazamos ligeramente el sensor entre distintas capturas, cada imagen aporta muestras diferentes de la escena. En el ejemplo se representan distintos desplazamientos de un píxel en horizontal y vertical, pero en la práctica pueden ser desplazamientos subpíxel. Después de registrar o alinear globalmente las imágenes, podemos combinar esas muestras complementarias para rellenar posiciones que una única captura no habría observado directamente.

En el caso de sensores con patrón Bayer, esta estrategia también puede ayudar a reducir la cantidad de información que debe inferirse durante el demosaicing. En lugar de estimar todos los colores que faltan únicamente a partir de una imagen, varias capturas desplazadas pueden aportar observaciones adicionales de esos canales de color.

La idea fundamental es, por tanto, que aquí sí podemos recuperar información adicional porque no estamos trabajando únicamente con una imagen de baja resolución: disponemos de varias observaciones diferentes de la misma escena.

Summary

- Key points:
  - **Subsampling an image** can cause aliasing. Better is to blur ("pre-filter") to remove high frequencies then downsample.
  - If you repeatedly blur and downsample by 2x, you get a **Gaussian Pyramid**.
  - From this Gaussian Pyramid, we can compute the **Laplacian Pyramid** (that can be useful in image blending or image compression).
  - **Upsampling an image** requires interpolation. This can be posed as convolution with a "reconstruction kernel".

En esta diapositiva resumimos las ideas principales de este bloque.

En primer lugar, hemos visto que reducir directamente el tamaño de una imagen mediante subsampling puede producir aliasing. Para evitarlo, antes de reducir la resolución conviene aplicar un filtro paso bajo que elimine o atenúe suficientemente las altas frecuencias que ya no podrán representarse correctamente después del downsampling.

Si repetimos sucesivamente este proceso de suavizado y reducción, obtenemos una pirámide gaussiana, que nos proporciona distintas representaciones de la misma imagen a diferentes escalas.

A partir de la pirámide gaussiana podemos construir una pirámide laplaciana, que representa principalmente la información de detalle presente entre escalas consecutivas. Como hemos visto, este tipo de representación resulta útil, por ejemplo, en aplicaciones de blending y compresión.

Finalmente, hemos estudiado el problema opuesto: aumentar el tamaño de una imagen. En este caso necesitamos estimar nuevos valores en posiciones donde originalmente no teníamos muestras, y para ello utilizamos interpolación. Esta interpolación puede interpretarse como una reconstrucción mediante un determinado kernel.

Homework

- Check the following Steve Seitz's videos to review some of the ideas presented so far in the course:
  - "Images in 5 minutes: The Case of the Splotched Van Gogh, Part 1" (https://www.youtube.com/watch?v=xUzhKqf22mY&list=PLWfDJ5nla8UpwShx-lzLJqcp575fKpsSO&index=5) (4'37")
  - "Image filtering in 5 minutes: The Case of the Splotched Van Gogh, Part 2" (https://www.youtube.com/watch?v=6v8dNtknOSM&list=PLWfDJ5nla8UpwShx-lzLJqcp575fKpsSO&index=6) (5'06")
  - "Fourier Transform in 5 minutes: The Case of the Splotched Van Gogh, Part 3" (https://www.youtube.com/watch?v=JciZYrh36LY&list=PLWfDJ5nla8UpwShx-lzLJqcp575fKpsSO&index=7) (8'09")
  - "Color Perception in 5 minutes" (https://www.youtube.com/watch?v=cM2wvHtVcM8&list=PLWfDJ5nla8UpwShx-lzLJqcp575fKpsSO&index=4) (7'49")
  - "Interpolation in 5 minutes" (https://www.youtube.com/watch?v=Xj129kA3Ci0&list=PLWfDJ5nla8UpwShx-lzLJqcp575fKpsSO&index=8) (5'30")

Para terminar, os dejo como trabajo una serie de vídeos cortos de Steve Seitz que permiten repasar de forma muy visual e intuitiva varias de las ideas que hemos visto en este bloque: representación de imágenes, filtrado, aliasing, transformada de Fourier, percepción del color e interpolación. Son vídeos muy breves y están pensados precisamente como complemento y repaso de estos conceptos.

Pyramids

Pablo Mesejo
pmesejo@go.ugr.es
Universidad de Granada
Departamento de Ciencias de la Computación e Inteligencia Artificial

